ECE – 6370

Lab – 4

ROM-Based Game Access Control and LFSR Timer on FPGA

Under the Supervision

of

Dr. Yuhua Chen

Department of Electrical and Computer Engineering

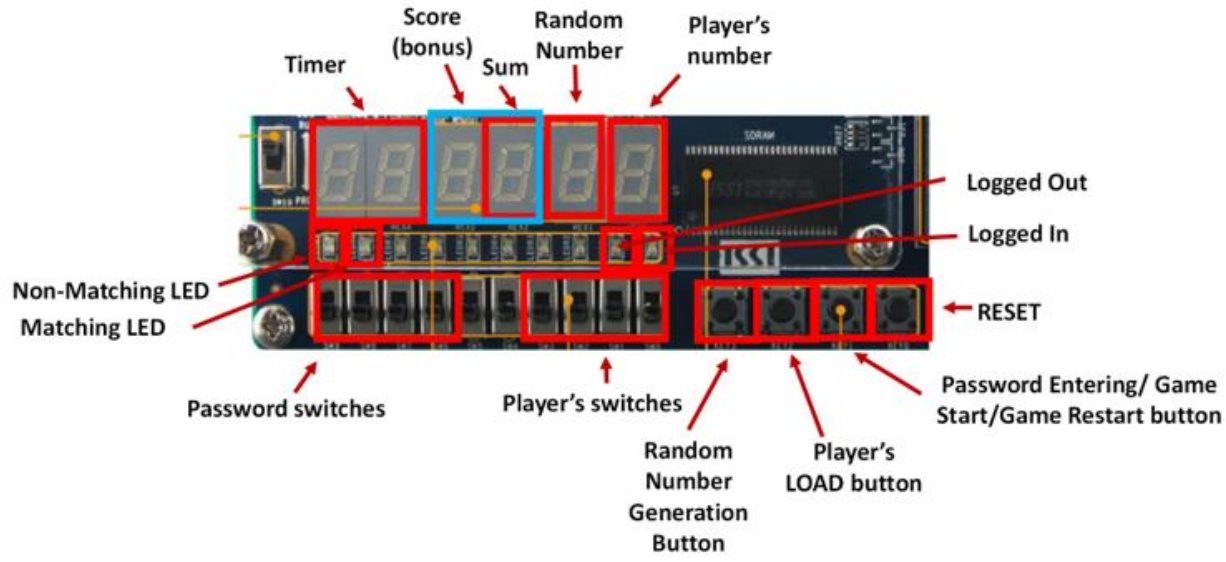
Affan Iqbal

Introduction:

Lab 4 extends the authenticated binary-sum game developed in Lab 3 by introducing two major architectural enhancements:

1. **ROM-Based Password Authentication** The 4-digit password is no longer hardcoded inside the Access Controller FSM. Instead, it is stored in a Quartus-generated synchronous ROM. The Access FSM retrieves each digit from the ROM one at a time using a multi-cycle read pipeline, ensuring correct timing with the registered ROM output.
2. **Separation of Access Control and Timer Control** The monolithic Access Controller from Lab 3 is split into two independent FSMs:
    - **access_fsm** — handles password entry, ROM-based verification, authentication, game start, and game-over transitions.
    - **timer_fsm** — handles timer reconfiguration, timer enable gating, RNG gating, and LOAD gating during gameplay.

These changes improve modularity, maintainability, and security while preserving the user interface and gameplay flow from Lab 3.



System Architecture

The system is organized into three major functional subsystems: **Authentication**, **Player Input & Random Number Control**, and **Game Logic with Timer Management**. Lab 3 builds on the foundation established in Lab 2 but extends the design by introducing hardware-based randomness and a timer-controlled gameplay mechanism. These additions transform the system from a simple authenticated input-output design into a fully sequenced digital game environment where access, input acceptance, random number generation, and gameplay duration are all governed by hardware logic.

The system now behaves even more like a real embedded platform: inputs are validated, game actions are enabled only in the correct states, and the player interacts with a time-limited challenge. The Access Controller FSM plays a central role by coordinating authentication, timer loading, game start, and game termination.

Architectural Overview

i. ROM-Based Authentication Subsystem

- User enters four digits using switches and the ENTER button.
- access_fsm captures the digits (p1–p4).
- access_fsm reads the stored password from ROM_PSWD using:

  check0_set → check0_cap

  check1_set → check1_cap

  check2_set → check2_cap

  check3_set → check3_cap
- Each ROM read takes two cycles because the ROM output is registered.
- If all digits match, the system transitions to authenticated mode.

ii. Timer Control Subsystem

imer_fsm handles:

- Timer reconfiguration after authentication
- Timer enable during gameplay
- RNG gating (rng_out)
- LOAD gating (load_out)

- Disabling gameplay on timeout

This separation ensures that authentication logic and gameplay timing logic operate

iii. Player Input & Random Number Control Subsystem

This subsystem replaces Player 2 from Lab 2 with a hardware-based True Random Number Generator (RNG). It includes:

- **Button Shapers** for the Player LOAD button and the ENTER button (password entry).
- **Load Register** for capturing the Player's 4-bit number only when authenticated.

iv. LFSR-Based Random Number Generator (Bonus Feature)

In Lab 3, the random number generator was implemented using a free-running 4-bit binary counter. For Lab 4, the RNG has been upgraded to a **4-bit Linear Feedback Shift Register (LFSR)** to reduce hardware cost and improve randomness quality.

The LFSR uses the primitive polynomial:

$$x^4 + x^3 + 1$$

with taps at bit3 and bit2. This produces a maximal-length sequence of 15 non-zero states.

- Key behaviors: When rng_gen = 0, the LFSR shifts and generates new values at 50 MHz.
- When rng_gen = 1, the output holds its current value.
- On reset, the output initializes to 0, and the first active cycle seeds the LFSR with 0001 to avoid the illegal all-zero state.
- This LFSR replaces the binary counter from Lab 3 and satisfies the bonus requirement for a cost-efficient RNG.

v. Logout Function (Bonus Feature)

Lab 4 introduces an optional logout feature. When the user is authenticated **and the timer is idle**, pressing the LOAD button logs the user out.

Behavior:

- logged_in LED turns OFF
- logged_out LED turns ON
- All gameplay functions (RNG, LOAD, timer) are disabled
- The system returns to the password entry state
- The user must re-enter the password to play again

This feature improves system usability and security by allowing the user to exit the authenticated state without resetting the entire FPGA.

vi. Game Logic & Timer Subsystem

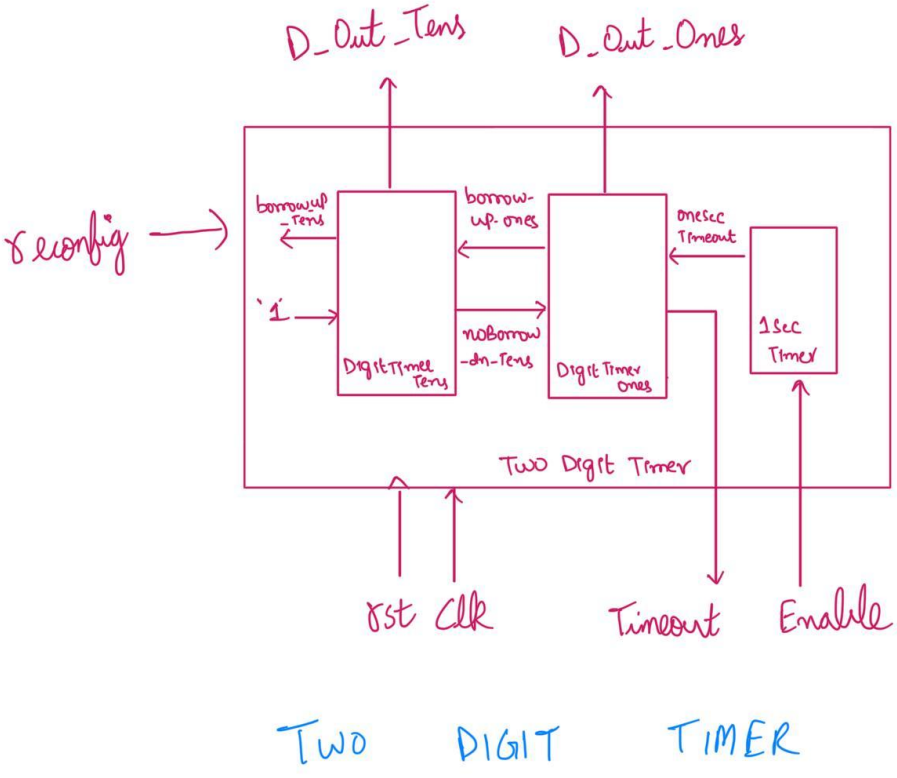
This subsystem performs the core game operations:

- **Binary Adder** computes the sum of the Player's number and the RNG-generated number.
- **Seven-Segment Decoders** display the Player's value, the RNG value, the sum, and the remaining game time.
- **Matching/Non-Matching LEDs** indicate whether the sum equals F (1111_2).
- **One-Second Timer** divides the 50 MHz clock to generate a 1 Hz pulse.
- **Two-Digit Countdown Timer** counts down from 99 to 00 using BCD digit modules with borrow-down chaining.
- **Timer-Controlled Game Flow**:
    - After authentication, the timer loads "99".
    - Pressing the GAME button starts the countdown.
    - When the timer reaches zero, all game inputs are disabled.
    - Pressing the GAME button again reloads the timer for a new round.

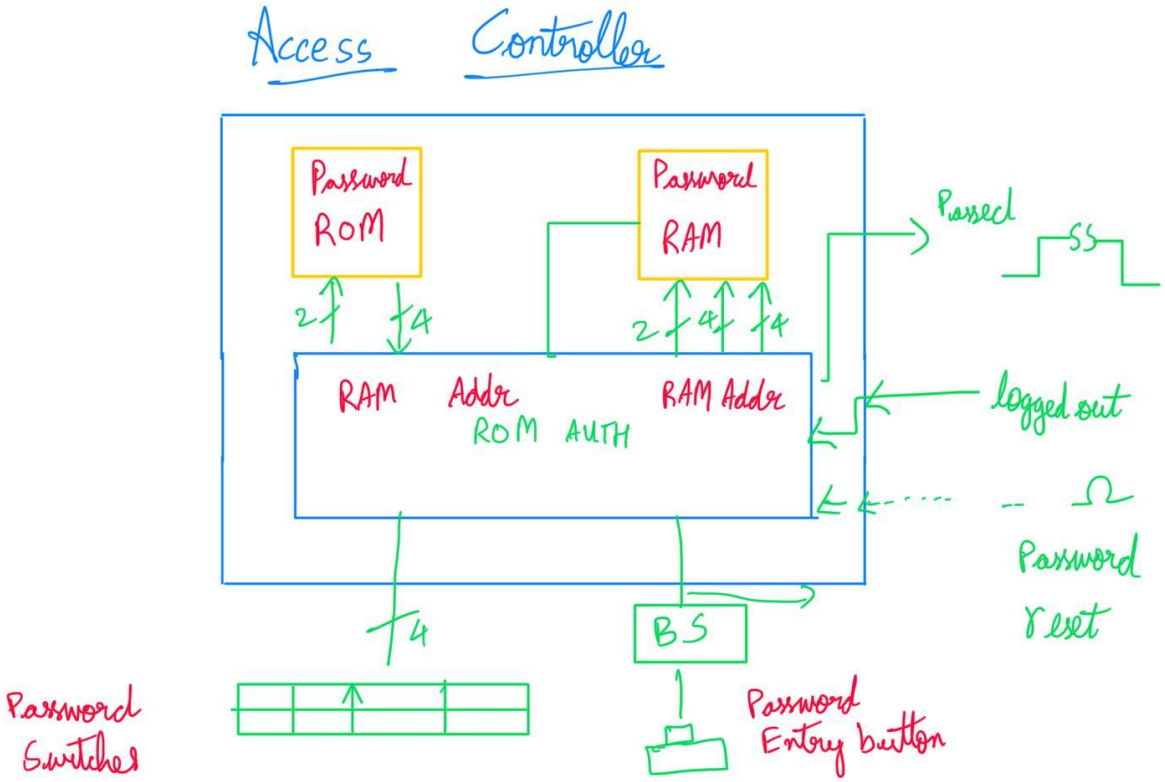
This subsystem ensures that gameplay is time-bounded and that the system transitions cleanly between game start, active play, and game-over states.

Block Diagrams

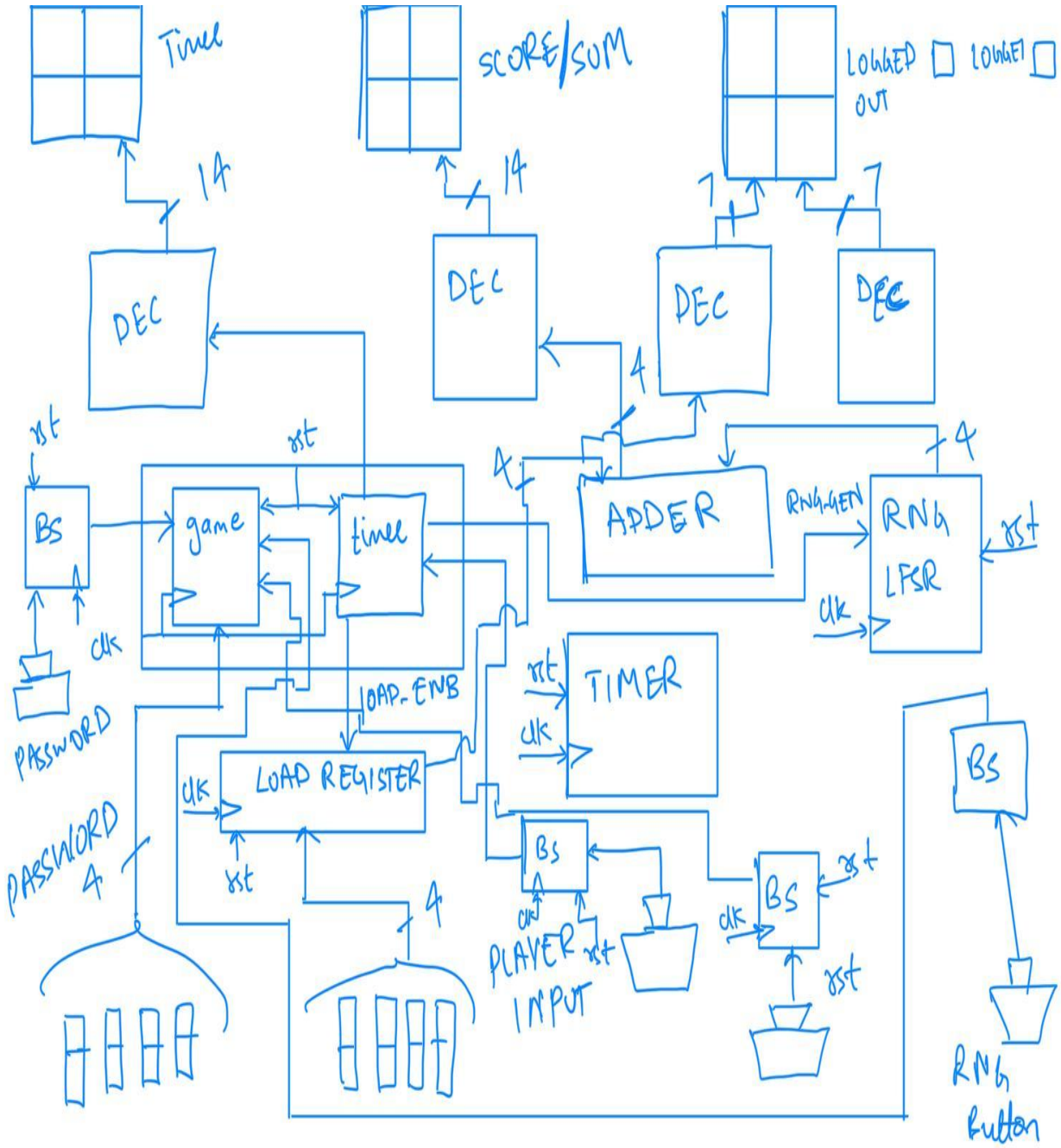
Timer Block Diagram

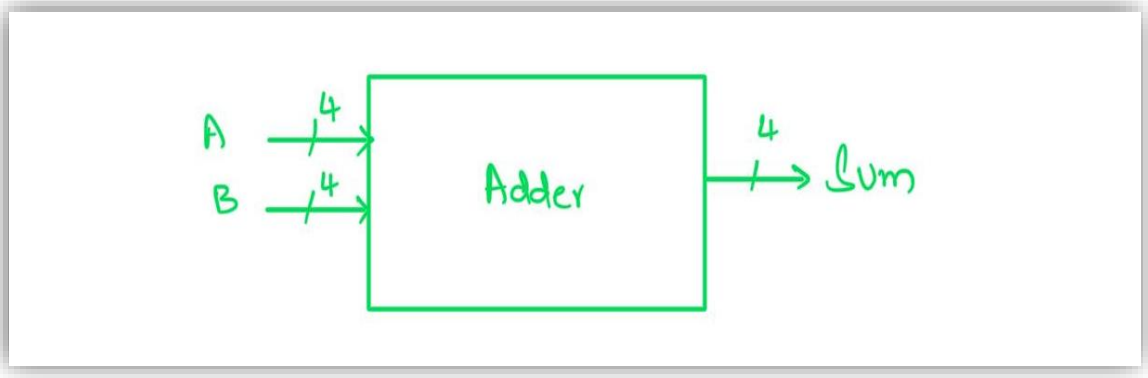


Access Controller Block Diagram

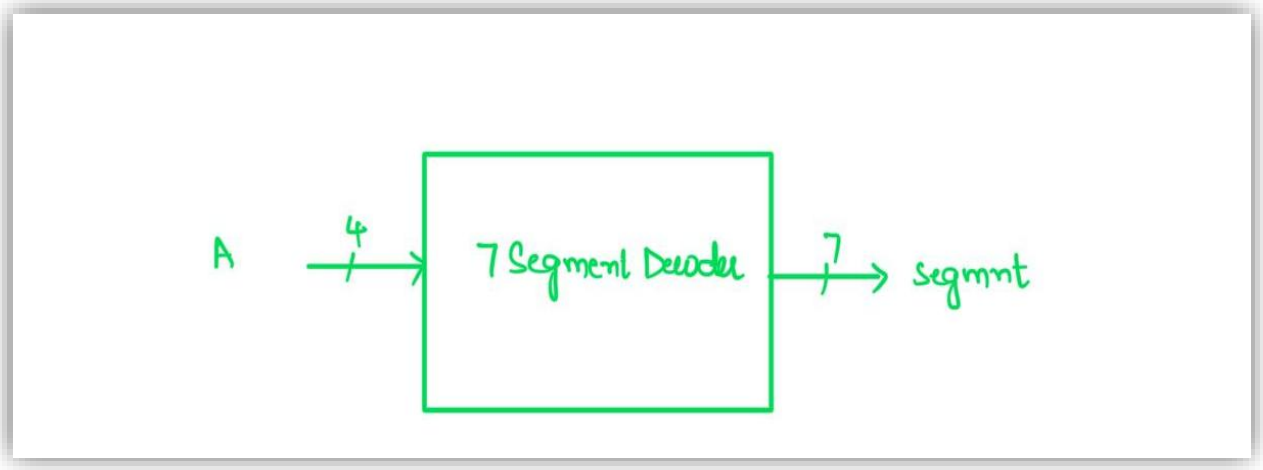


Top-Level System Architecture





Block Diagram of Adder



Block Diagram of 7-Segment



Block Diagram of Load Register

Module Description

Seven Segment Decoder

- Input: 4-bit number
- Output: 7-bit active-low pattern
- Used for Player value, RNG value, Sum, and Timer digits.

Four-Bit Adder

- Inputs: Player value, RNG value
- Output: 4-bit sum
- Drives the middle seven-segment display.

Button Shaper

- Converts noisy pushbutton input into a clean single-cycle pulse.
- Used for ENTER, LOAD, and GAME buttons.

Load Register

- Stores Player 1's 4-bit value only when authenticated and LOAD pulse is received.

LFSR-Based Random Number Generator (Updated Module)

The RNG module has been redesigned using a 4-bit Linear Feedback Shift Register (LFSR). The module uses taps at bit3 and bit2 to implement the primitive polynomial $x^4 + x^3 + 1$, producing a maximal-length sequence of 15 unique non-zero values.

Key Features:

- Active-low reset initializes the output to 0
- First active cycle seeds the LFSR with 0001
- rng_gen = 0 → LFSR runs
- rng_gen = 1 → output holds
- Fully synchronous with the 50 MHz system clock

One-Second Timer (Updated for Lab 4)

The one-second timer in Lab 4 is generated using a **hybrid timing structure** that incorporates the new **LFSR-based 1 ms timer**. Instead of using a binary counter to generate the 1 ms tick (as in Lab 3), Lab 4 replaces the 1 ms counter with a **4-bit Linear Feedback Shift Register (LFSR)** to reduce hardware cost and improve resource efficiency.

- The **LFSR-based 1 ms timer** produces a clean 1 ms timeout pulse using a maximal-length LFSR sequence.
- The 1 ms pulses feed into the existing **count-to-100** module to generate a 100 ms pulse.
- The 100 ms pulses feed into the **count-to-10** module to generate a 1 second pulse.
- The resulting 1 Hz signal drives the two-digit countdown timer used during gameplay.

Access Controller FSM

In Lab 4, the Access Controller is redesigned into a modular two-FSM architecture that separates authentication from gameplay timing. This improves clarity, maintainability, and aligns with real embedded system design practices.

A. access_fsm — Authentication & Game Flow

The access_fsm handles all user-driven control flow:

- Captures four password digits using the ENTER button
- Reads the stored password from the ROM using a multi-cycle set/capture pipeline
- Compares entered digits with ROM digits
- Outputs authentication status (authenticated, not_authenticated)
- Issues a timer reconfiguration pulse after successful login

- Controls game start, active gameplay, and game-over transitions

This FSM is responsible for **what the user is allowed to do**.

B. timer_fsm — Timer, RNG, and LOAD Gating

The timer_fsm manages all gameplay-related timing and gating:

- Loads the 99-second timer after authentication
- Enables the timer during gameplay
- Disables gameplay on timeout
- Gates the RNG button (rng_out)
- Gates the LOAD button (load_out)
- Ensures no gameplay inputs are accepted before login or after timeout

This FSM is responsible for **when gameplay is allowed**.

C. access_controller Module

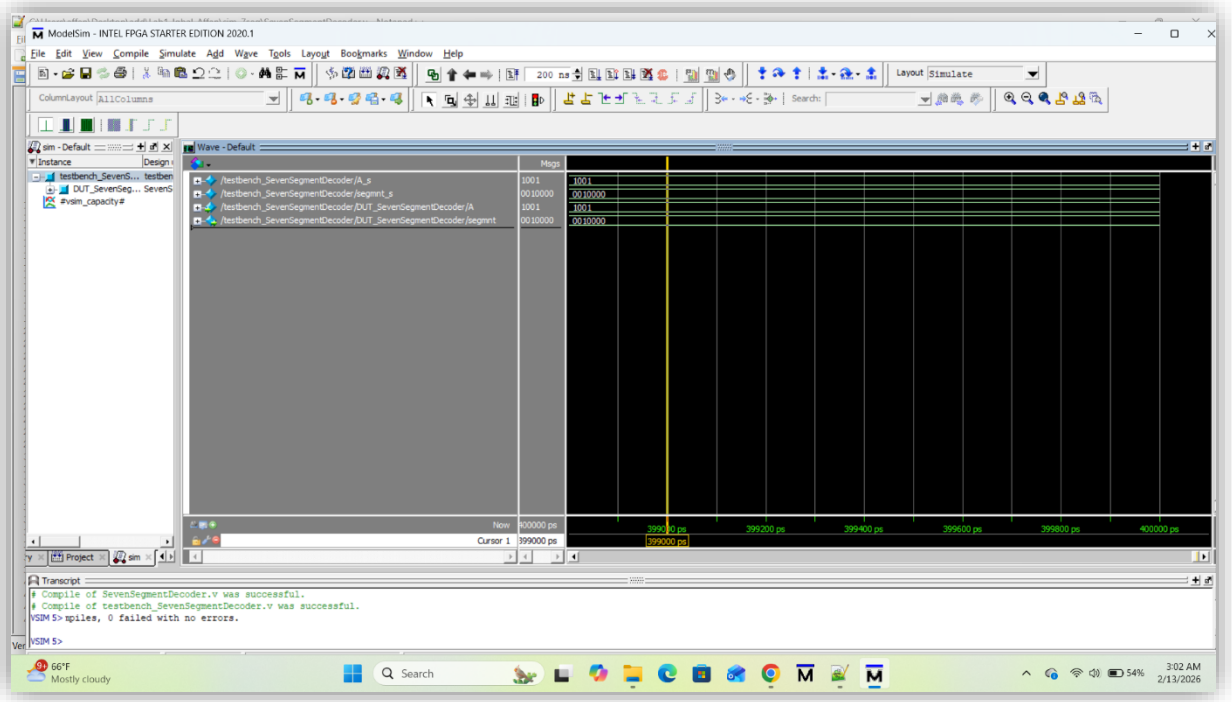
The access_controller module simply instantiates both FSMs and connects their handshake signals:

- start_game, game_active, game_over, reconfig_timer (from access_fsm)
- timer_enable, timer_reconfig, rng_out, load_out (from timer_fsm)

It exposes the same external interface as Lab 3, ensuring backward compatibility with the top-level module.

Simulation Results

- Seven-segment decoder



The SevenSegmentDecoder was tested for all hexadecimal inputs from 0 to F. The waveform shows that each input produces the correct active-low seven-segment pattern. The output updates immediately whenever the input changes, confirming correct combinational behavior.

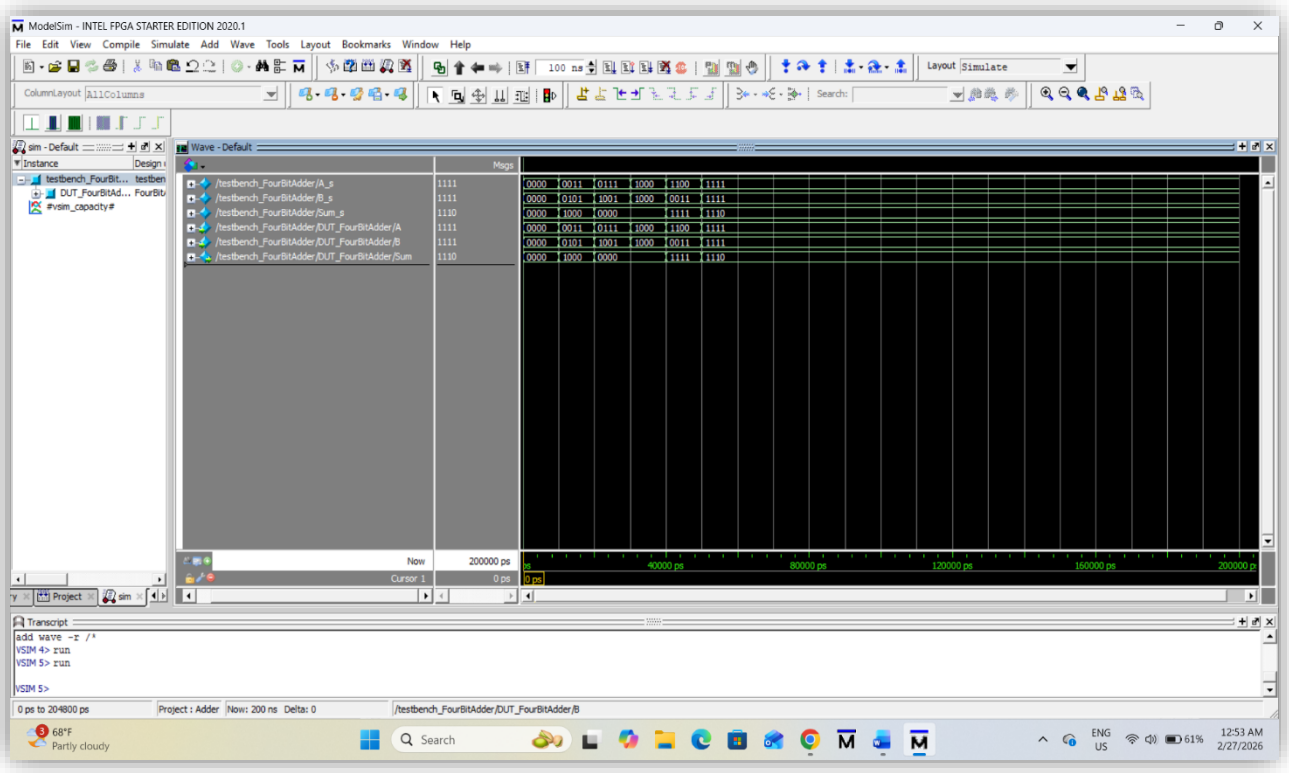
- **Input tested:** A = 1001 (decimal 9)
- **Observed output:** 0010000
- **Meaning:** Correct active-low seven-segment pattern for digit **9**
- **Testbench vs DUT:**

  segment_s (expected) = 0010000

  segment (actual DUT output) = 0010000

- Both match exactly

Four-Bit Adder



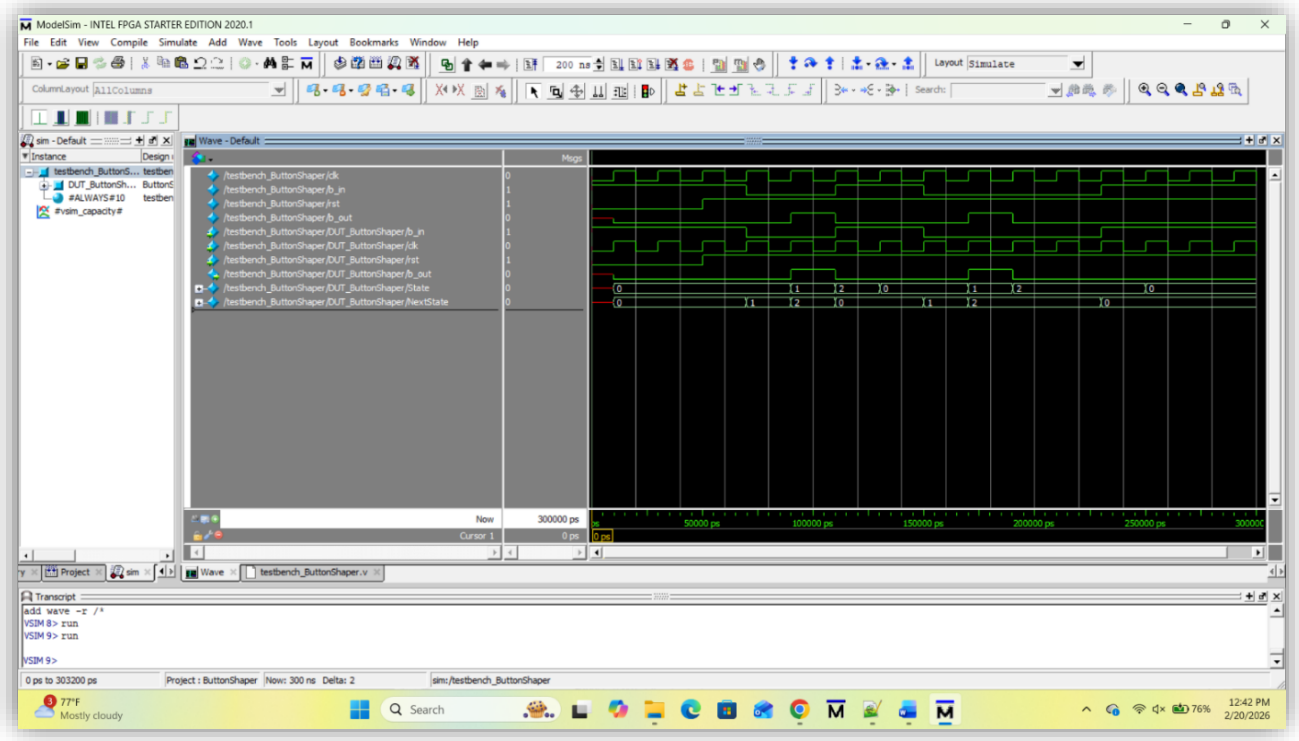
The FourBitAdder was tested with both matching and non-matching input pairs. The simulation shows correct 4-bit addition, including wrap-around behavior for values exceeding 15. The sum output updates instantly and matches the expected results for all tested cases.

Inputs tested: A = 1111, B = 1111

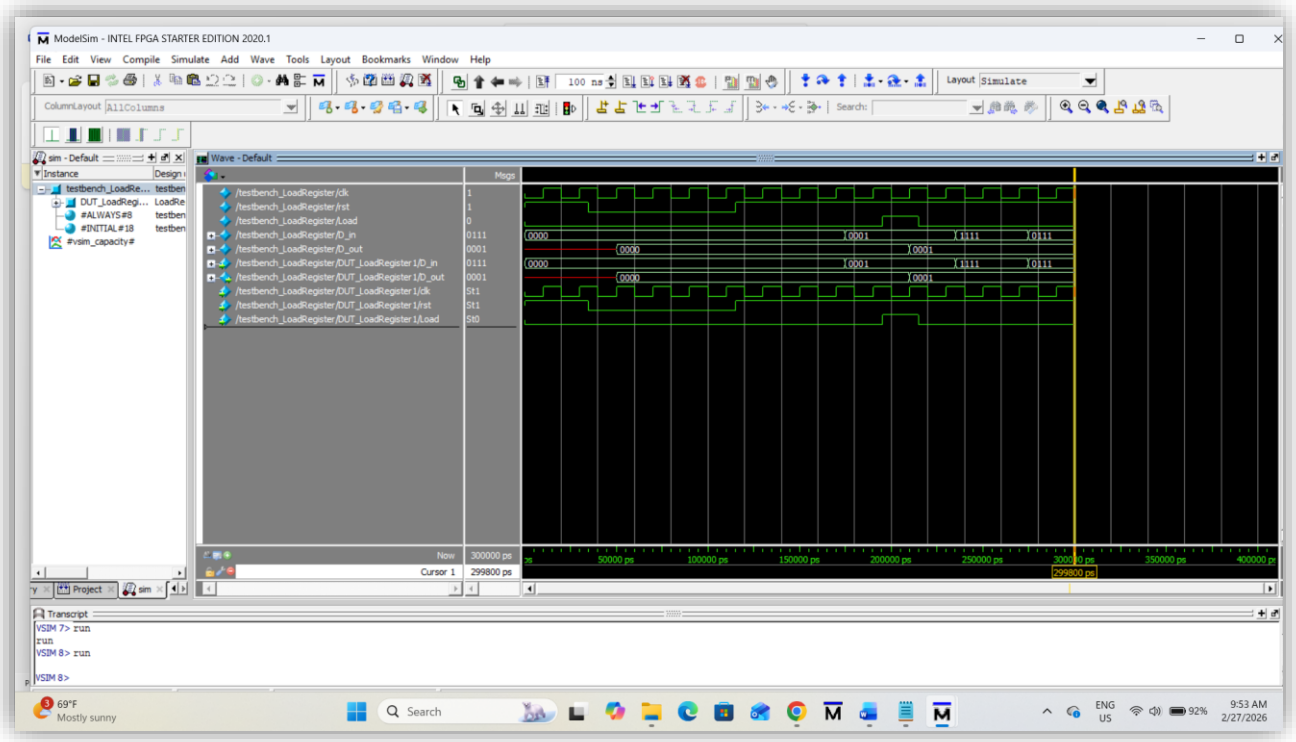
Observed output: SUM = 1110

- **Button Shaper Simulation**

The waveform shows the Button Shaper module responding correctly to the testbench inputs. After reset is released, each time the button input (b_in) goes low, the FSM transitions from the INIT state to the PULSE state, producing a single-cycle high output on b_out. The FSM then moves into the WAIT state until the button is released, after which it returns to INIT and becomes ready for the next press. The internal State and NextState signals in the waveform match the expected behavior, confirming that the module generates one clean pulse per button press and ignores long presses.

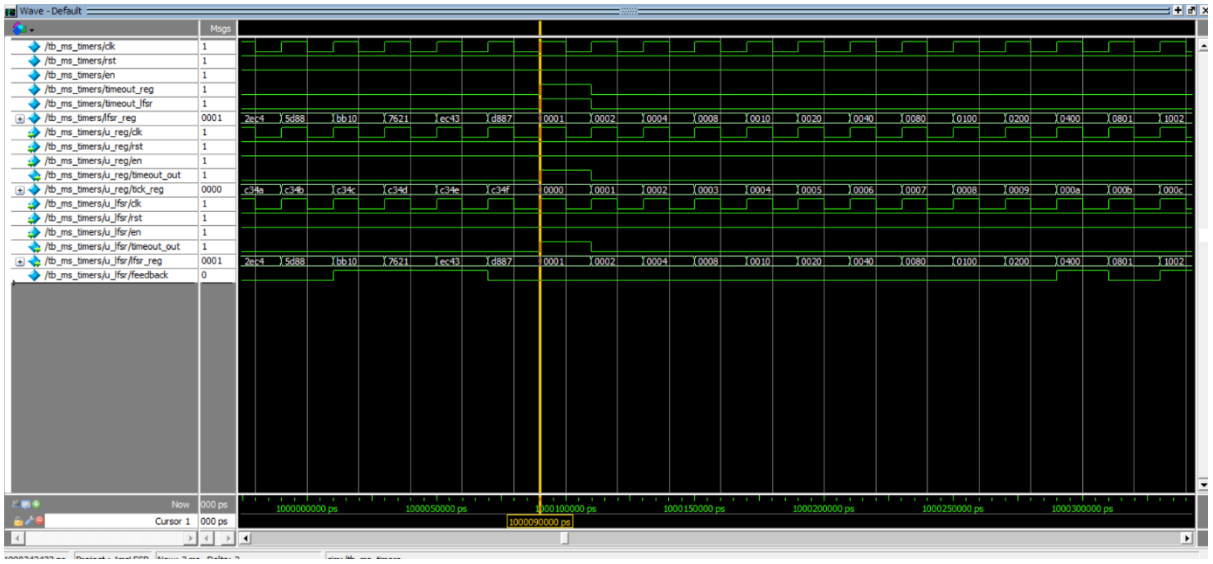


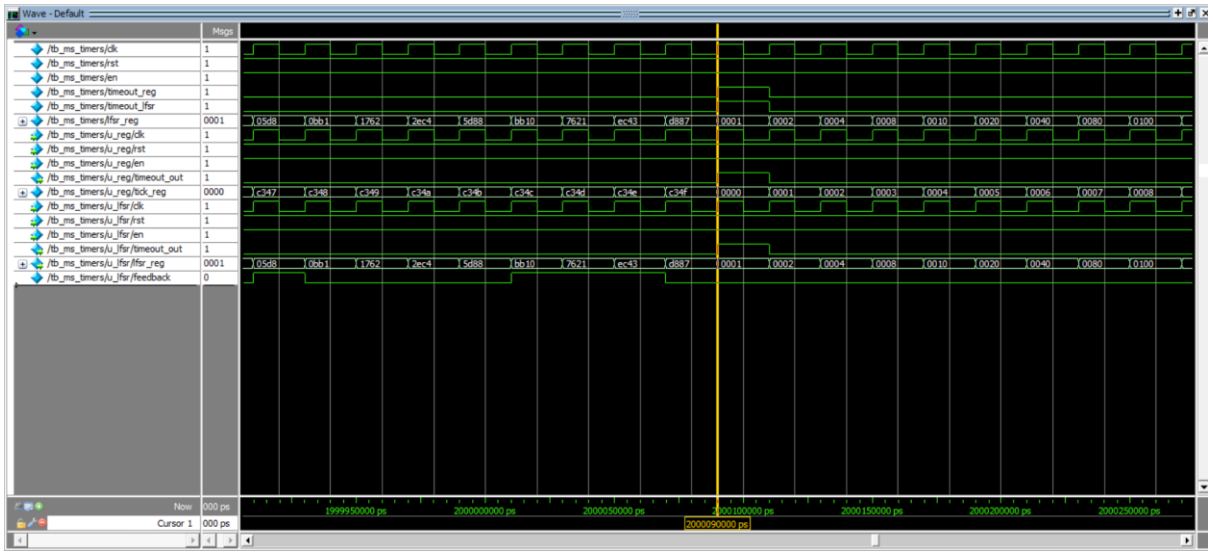
Load Register Simulation



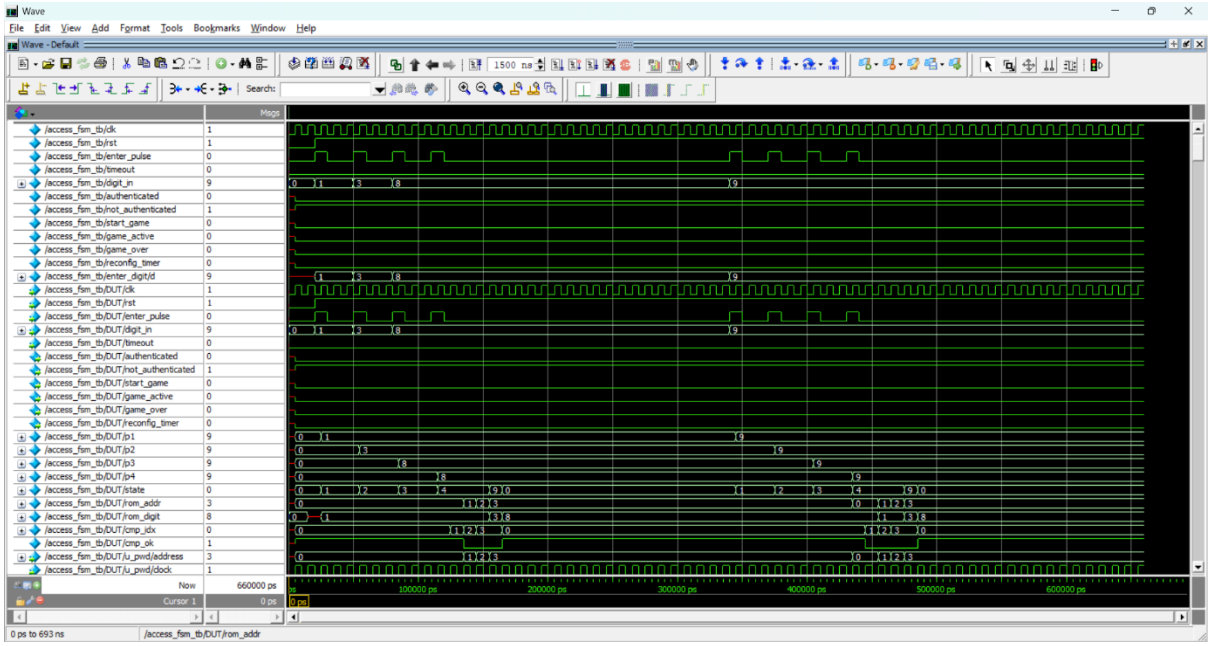
Load Register simulation showing the register updating only on a valid Load pulse while holding its value otherwise.

- **1ms_timer.v**


Wave - Default
Msgs
/tb_ms_timers/clk
/tb_ms_timers/rst
/tb_ms_timers/en
/tb_ms_timers/timeout_reg
/tb_ms_timers/timeout_lfsr
/tb_ms_timers/lfsr_reg
/tb_ms_timers/u_reg/clk
/tb_ms_timers/u_reg/rst
/tb_ms_timers/u_reg/en
/tb_ms_timers/u_reg/timeout_out
/tb_ms_timers/u_reg/tick_reg
/tb_ms_timers/u_lfsr/clk
/tb_ms_timers/u_lfsr/rst
/tb_ms_timers/u_lfsr/en
/tb_ms_timers/u_lfsr/timeout_out
/tb_ms_timers/u_lfsr/lfsr_reg
/tb_ms_timers/u_lfsr/feedback
Now
Cursor 1
1000090000 ps

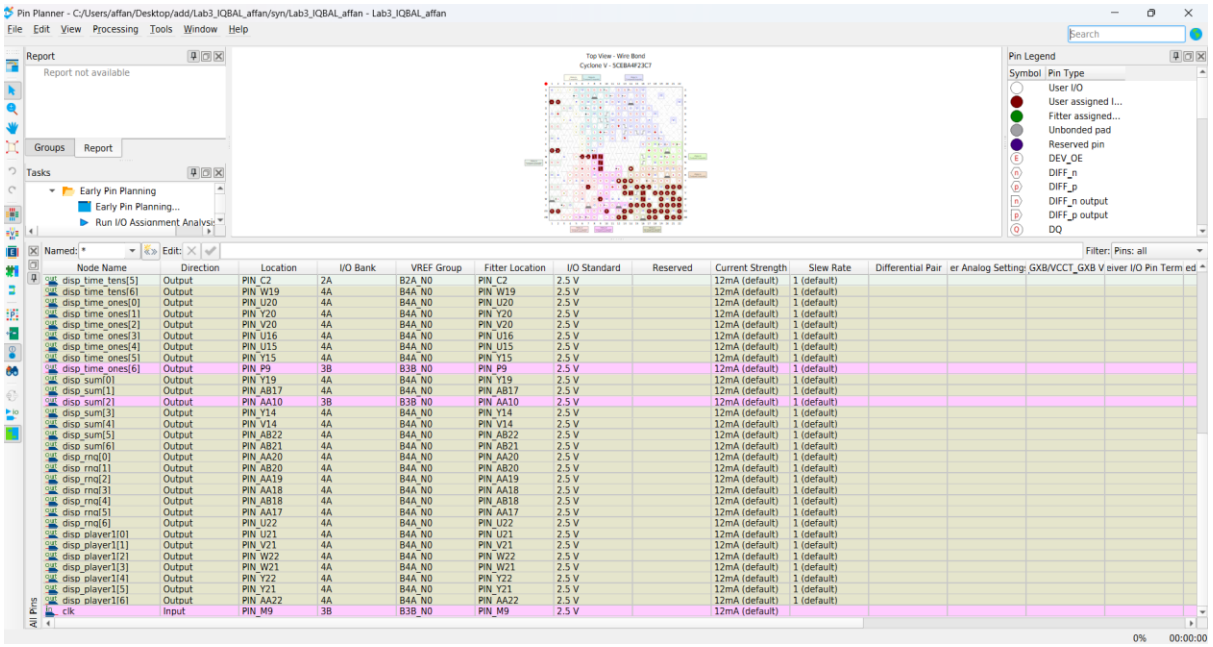

Wave - Default
Msgs
/tb_ms_timers/clk
/tb_ms_timers/rst
/tb_ms_timers/en
/tb_ms_timers/timeout_reg
/tb_ms_timers/timeout_lfsr
/tb_ms_timers/lfsr_reg
/tb_ms_timers/u_reg/clk
/tb_ms_timers/u_reg/rst
/tb_ms_timers/u_reg/en
/tb_ms_timers/u_reg/timeout_out
/tb_ms_timers/u_reg/tick_reg
/tb_ms_timers/u_lfsr/clk
/tb_ms_timers/u_lfsr/rst
/tb_ms_timers/u_lfsr/en
/tb_ms_timers/u_lfsr/timeout_out
/tb_ms_timers/u_lfsr/lfsr_reg
/tb_ms_timers/u_lfsr/feedback
Now
Cursor 1
2000090000 ps

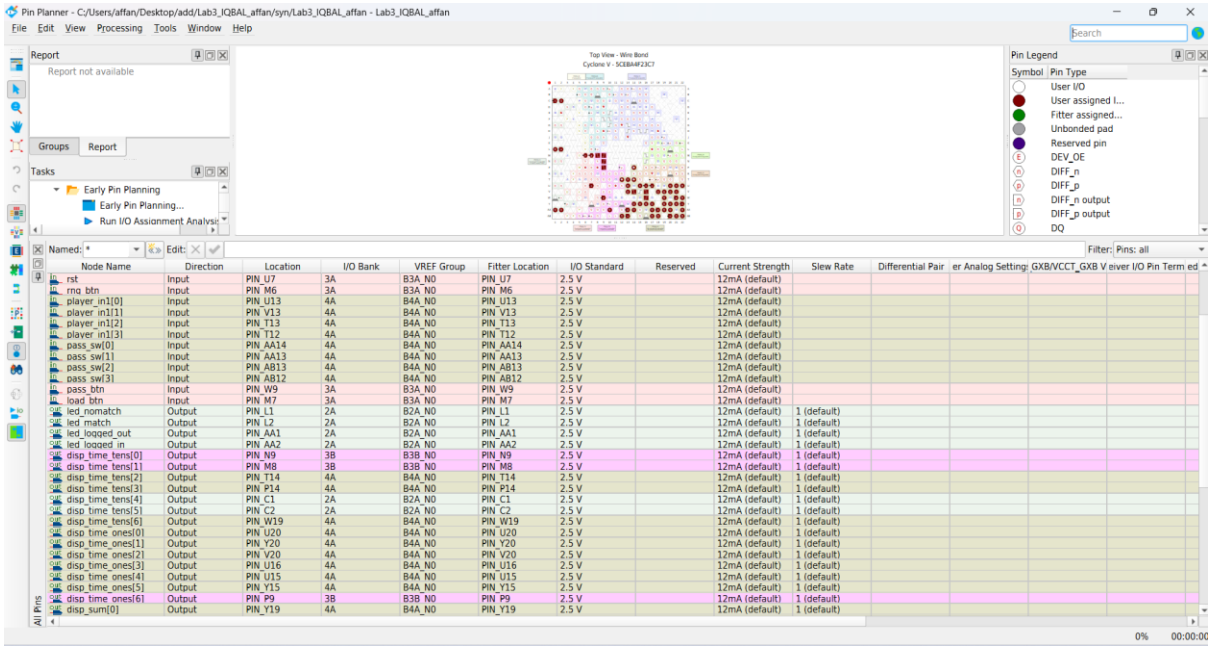
- **Access_FSM Simulation**



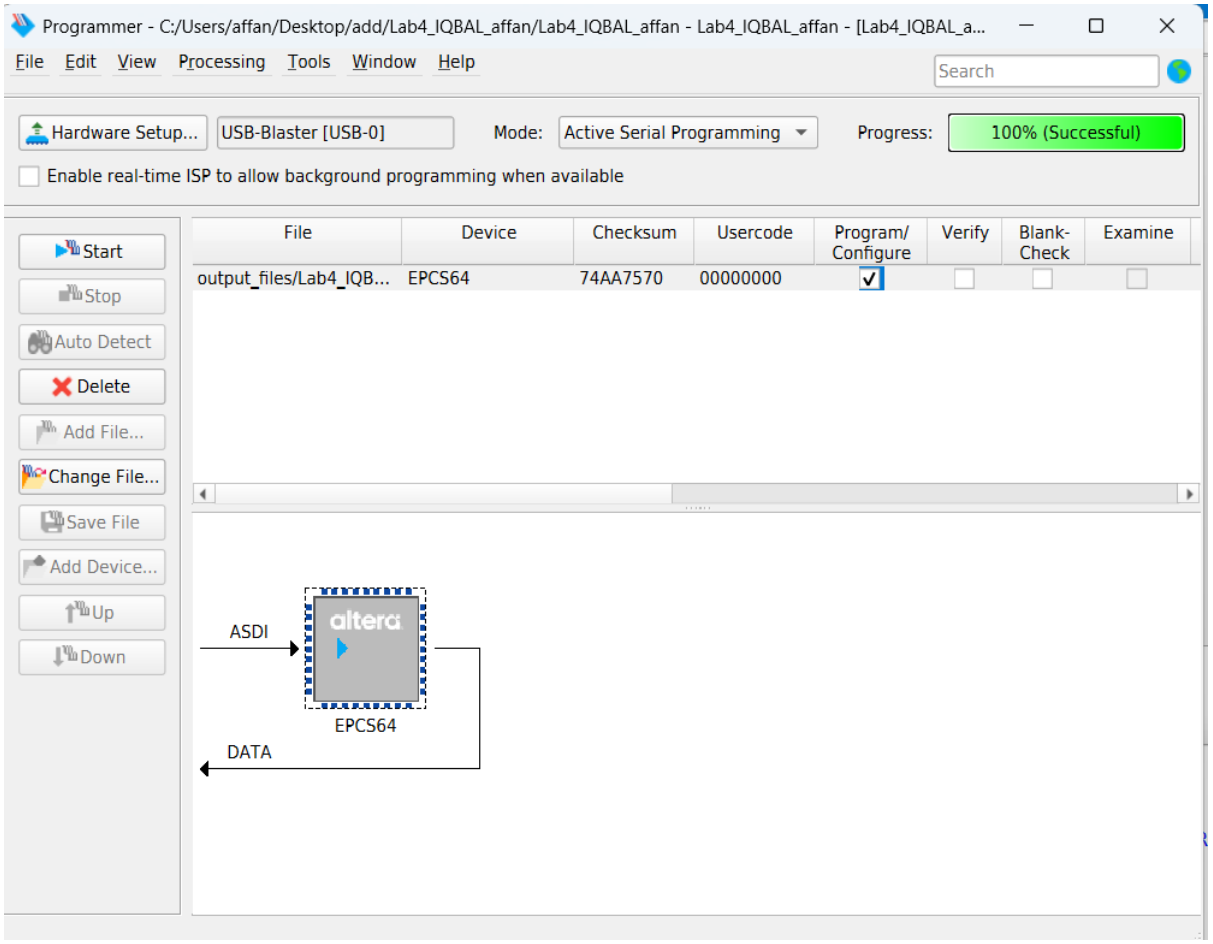
This waveform shows the Access FSM sequentially reading the stored password digits from the synchronous ROM. The ROM addresses 0, 1, 2, and 4 are accessed in order, returning the values **1**, **3**, **8**, and **8**, which correspond to the correct password **1388**. Each digit is captured one cycle after its address is issued, demonstrating the required multi-cycle ROM read pipeline. The captured ROM digits match the user-entered digits, confirming successful authentication.

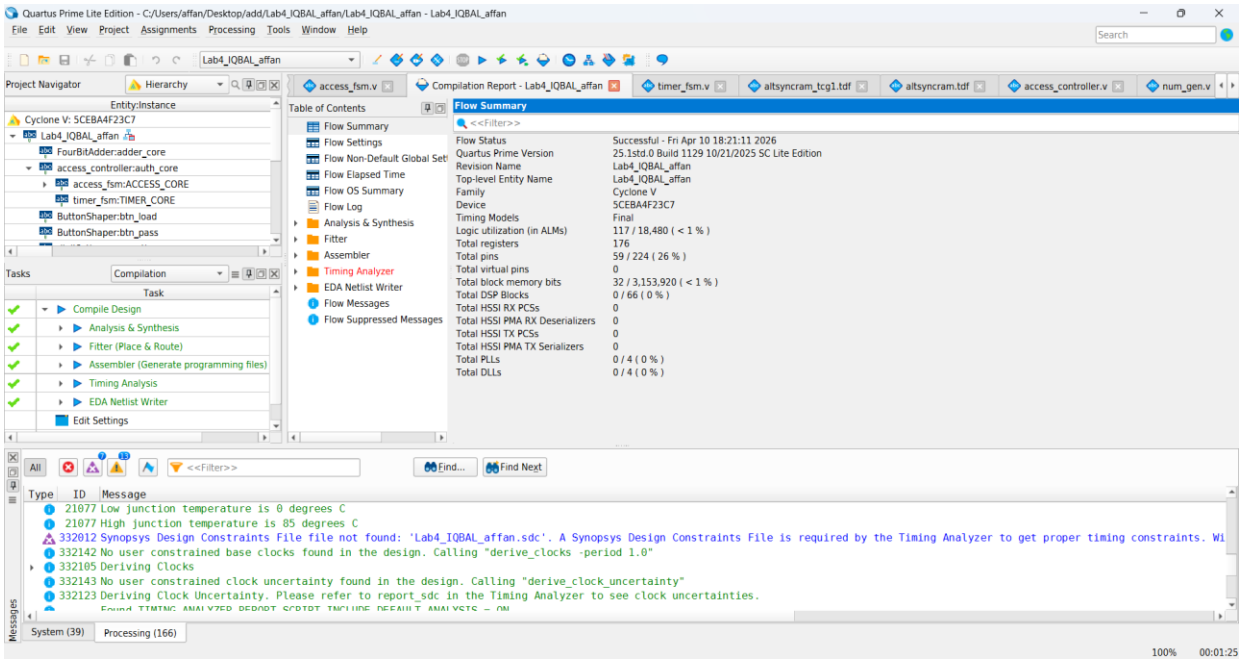
Pin Planner for Lab 4 Top Module





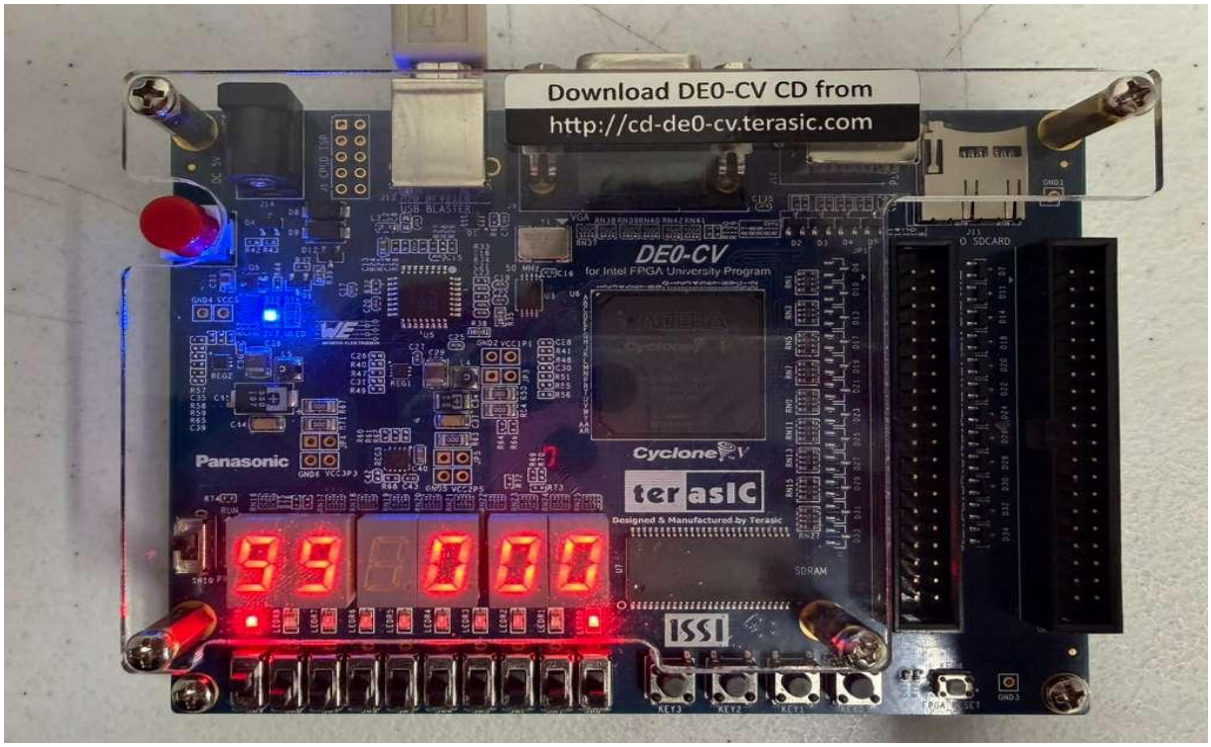
Successfully Compiled and Loaded Code on FPGA board


Programmer - C:/Users/affan/Desktop/add/Lab4_IQBAL_affan/Lab4_IQBAL_affan - Lab4_IQBAL_affan - [Lab4_IQBAL_a...
File Edit View Processing Tools Window Help
Search
Hardware Setup...
USB-Blaster [USB-0]
Mode: Active Serial Programming
Progress: 100% (Successful)
Enable real-time ISP to allow background programming when available
Start
Stop
Auto Detect
Delete
Add File...
Change File...
Save File
Add Device...
Up
Down
File
Device
Checksum
Usercode
Program/ Configure
Verify
Blank-Check
Examine
output_files/Lab4_IQB...
EPCS64
74AA7570
00000000
ASDI
altera
EPCS64
DATA

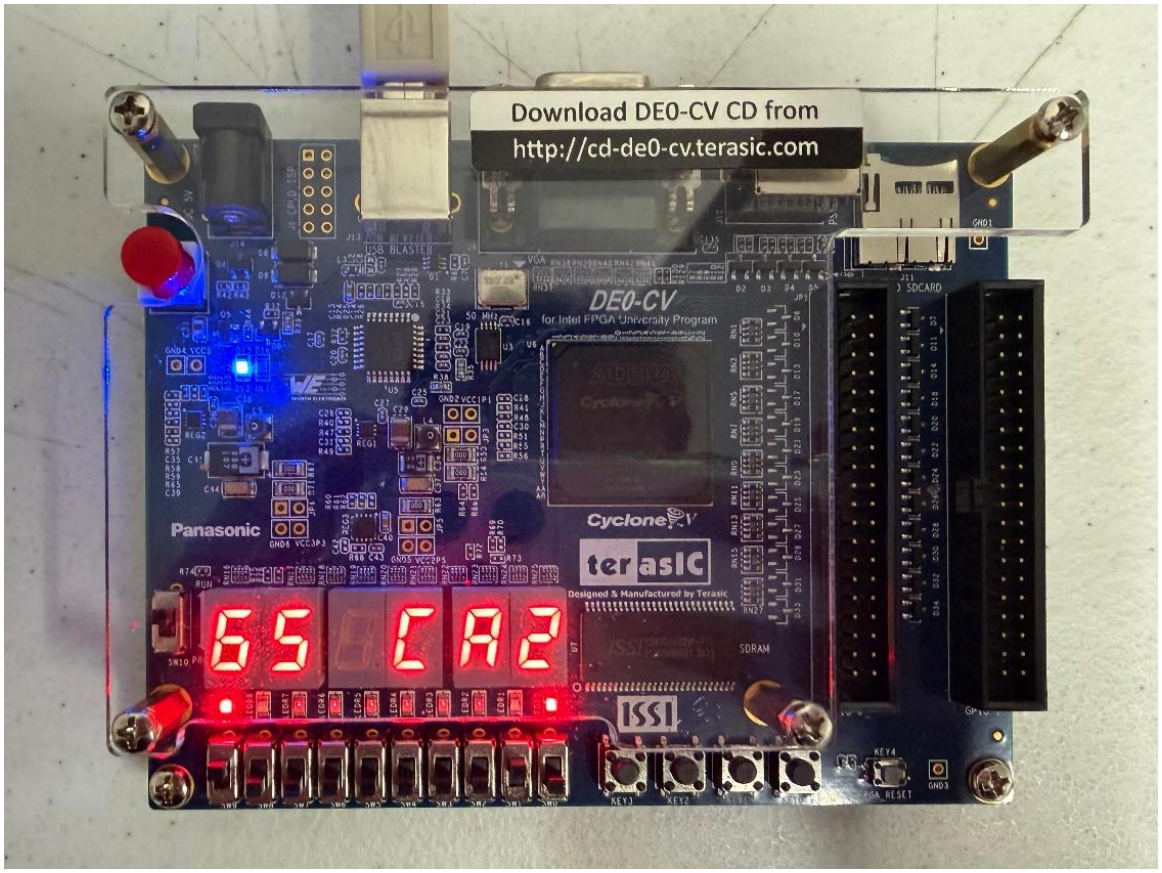

Quartus Prime Lite Edition - C:/Users/affan/Desktop/add/Lab4_IQBAL_affan/Lab4_IQBAL_affan - Lab4_IQBAL_affan
File Edit View Project Assignments Processing Tools Window Help
Flow Summary
Flow Status Successful - Fri Apr 10 18:21:11 2026
Quartus Prime Version 25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name Lab4_IQBAL_affan
Top-level Entity Name Lab4_IQBAL_affan
Family Cyclone V
Device 5CEBA4F23C7
Timing Models Final
Logic utilization (in ALMs) 117 / 18,480 (< 1 %)
Total registers 176
Total pins 59 / 224 (26 %)
Total virtual pins 0
Total block memory bits 32 / 3,153,920 (< 1 %)
Total DSP Blocks 0 / 66 (0 %)
Total HSSI RX PCSs 0
Total HSSI PMA RX Deserializers 0
Total HSSI TX PCSs 0
Total HSSI PMA TX Serializers 0
Total PLLs 0 / 4 (0 %)
Total DLLs 0 / 4 (0 %)
21077 Low junction temperature is 0 degrees C
21077 High junction temperature is 85 degrees C
332012 Synopsys Design Constraints File file not found: 'Lab4_IQBAL_affan.sdc'. A Synopsys Design Constraints File is required by the Timing Analyzer to get proper timing constraints.
332142 No user constrained base clocks found in the design. Calling "derive_clocks -period 1.0"
332105 Deriving Clocks
332143 No user constrained clock uncertainty found in the design. Calling "derive_clock_uncertainty"
332123 Deriving Clock Uncertainty. Please refer to report_sdc in the Timing Analyzer to see clock uncertainties.

FPGA Board Testing Results

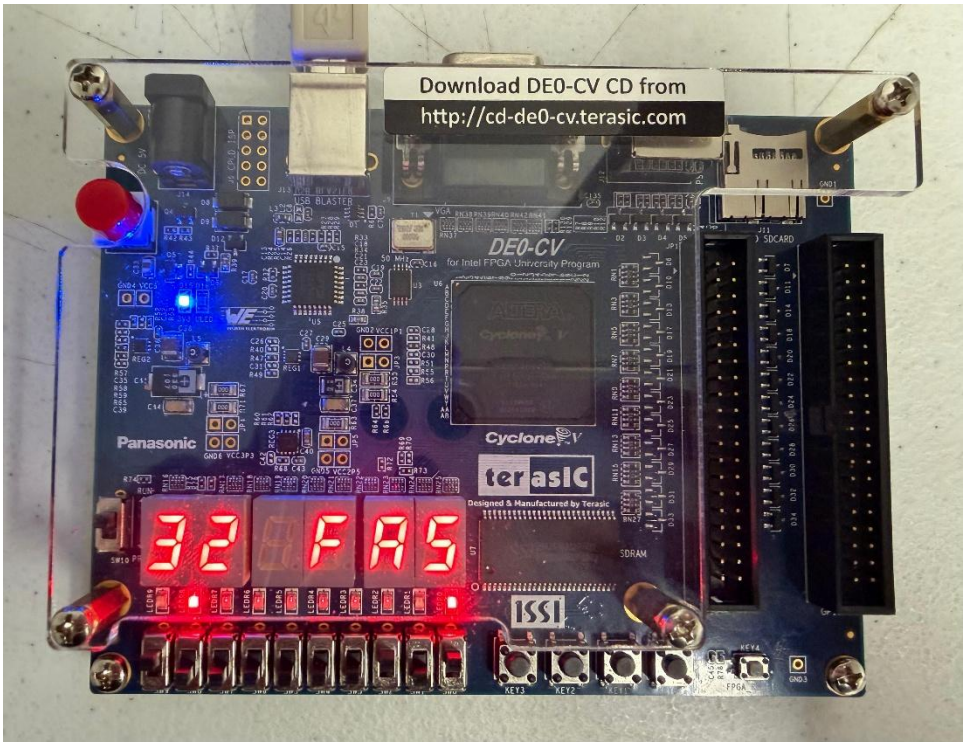
After Authentication, waiting for timer to start to play the game



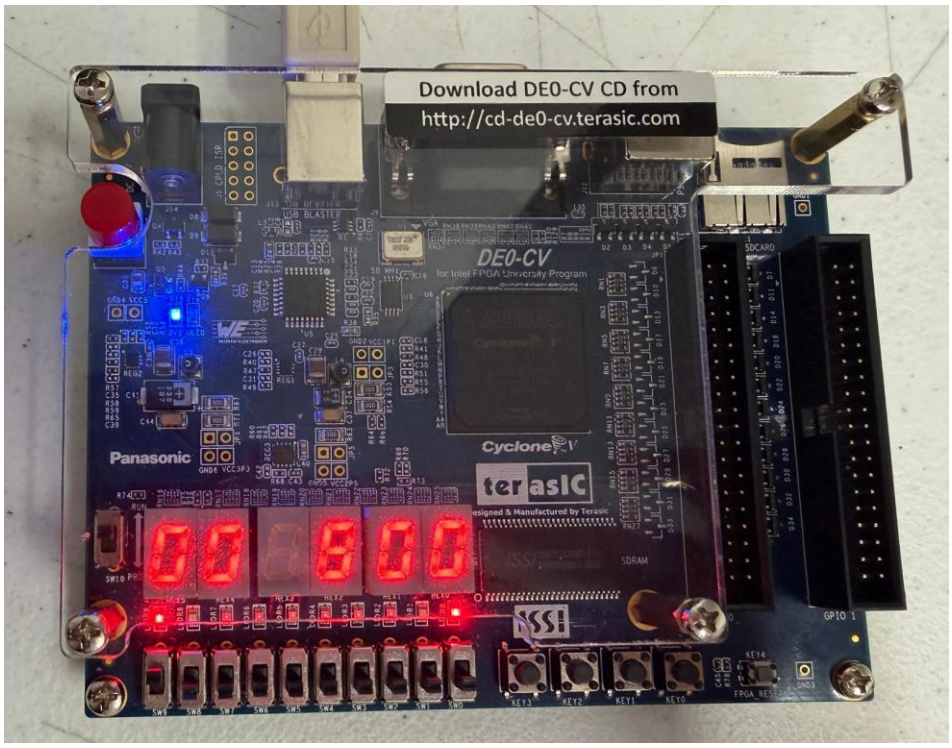
Timer is started:



The sum adds up to F, matching LED turns ON



Scoring After Game: The player has scored an 8



Conclusion

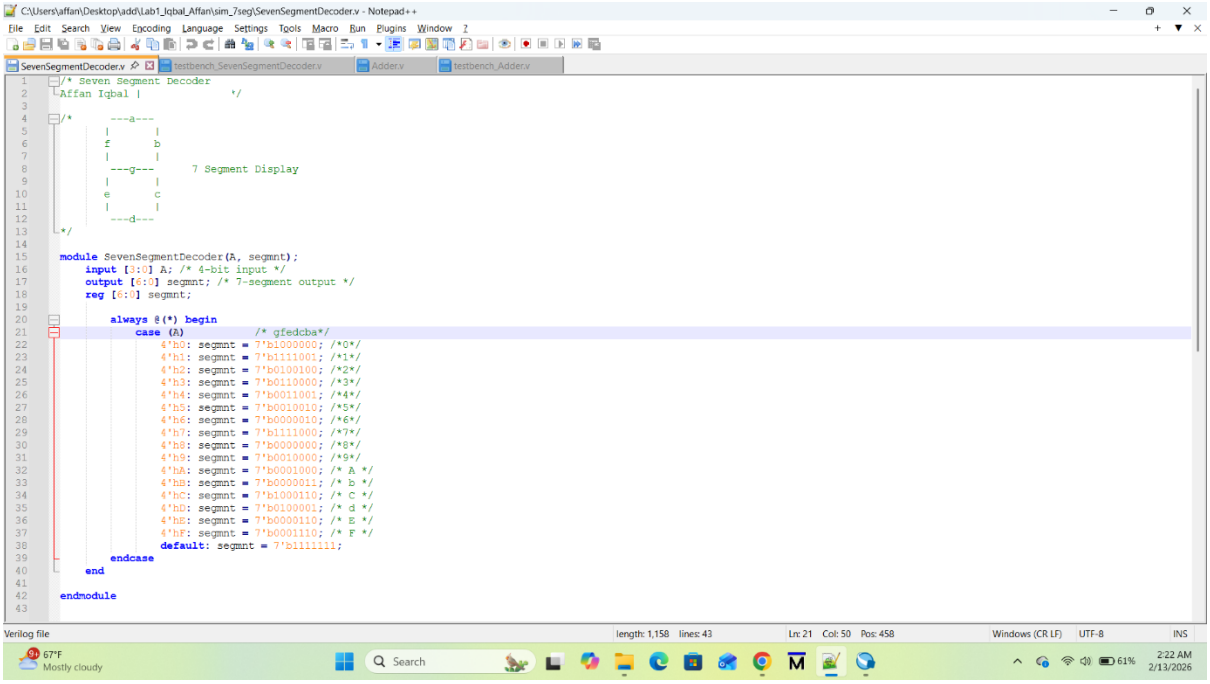
In this lab, I extended my authenticated binary-sum game by adding a hardware-based random number generator and a 99-second countdown timer. The RNG allowed the FPGA to act as the second player by generating unpredictable 4-bit values based on how long I held the button. The timer added a new layer of control by limiting gameplay and disabling all inputs once time ran out.

Through simulation, I verified that each module RNG, timer, button shapers, load registers, and the updated Access Controller worked correctly. After programming the FPGA, the hardware behaved exactly as expected: the system authenticated properly, the timer loaded and counted down, the RNG produced different values on each button release, and the game stopped accepting inputs when the timer reached zero.

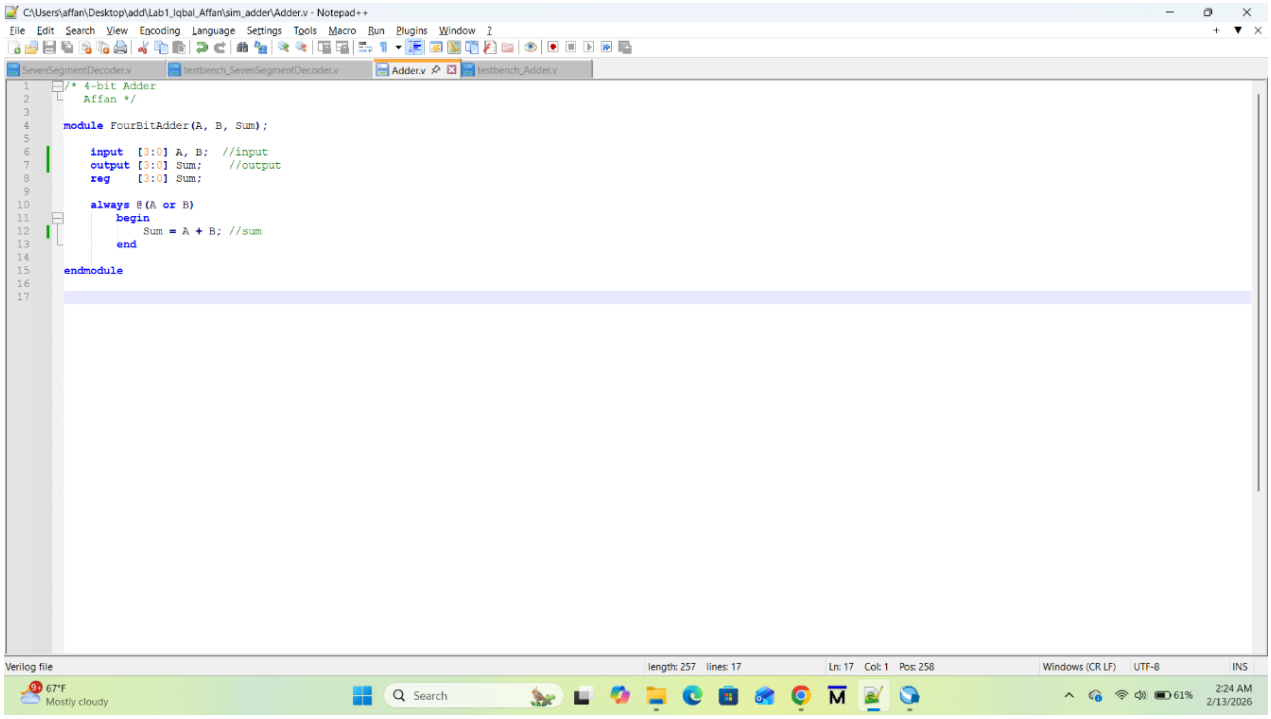
Overall, this lab helped me understand how multiple synchronous and asynchronous hardware components can be combined to create a complete, time-controlled digital system. It also showed how adding randomness and timing transforms a simple design into a more interactive and realistic FPGA-based game.

Appendix

Verilog Module for Seven Segment Decoder (SevenSegmentDecoder.v)

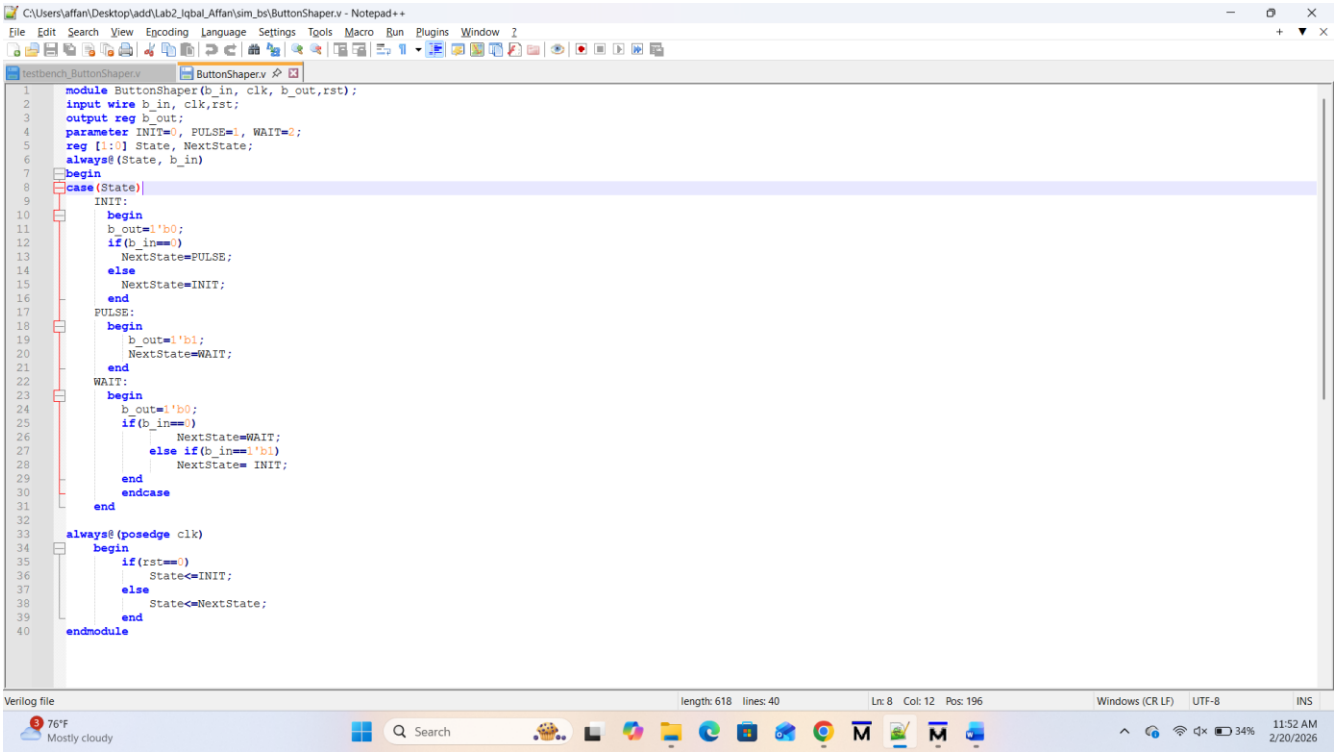


Verilog Module Adder (Adder.v)

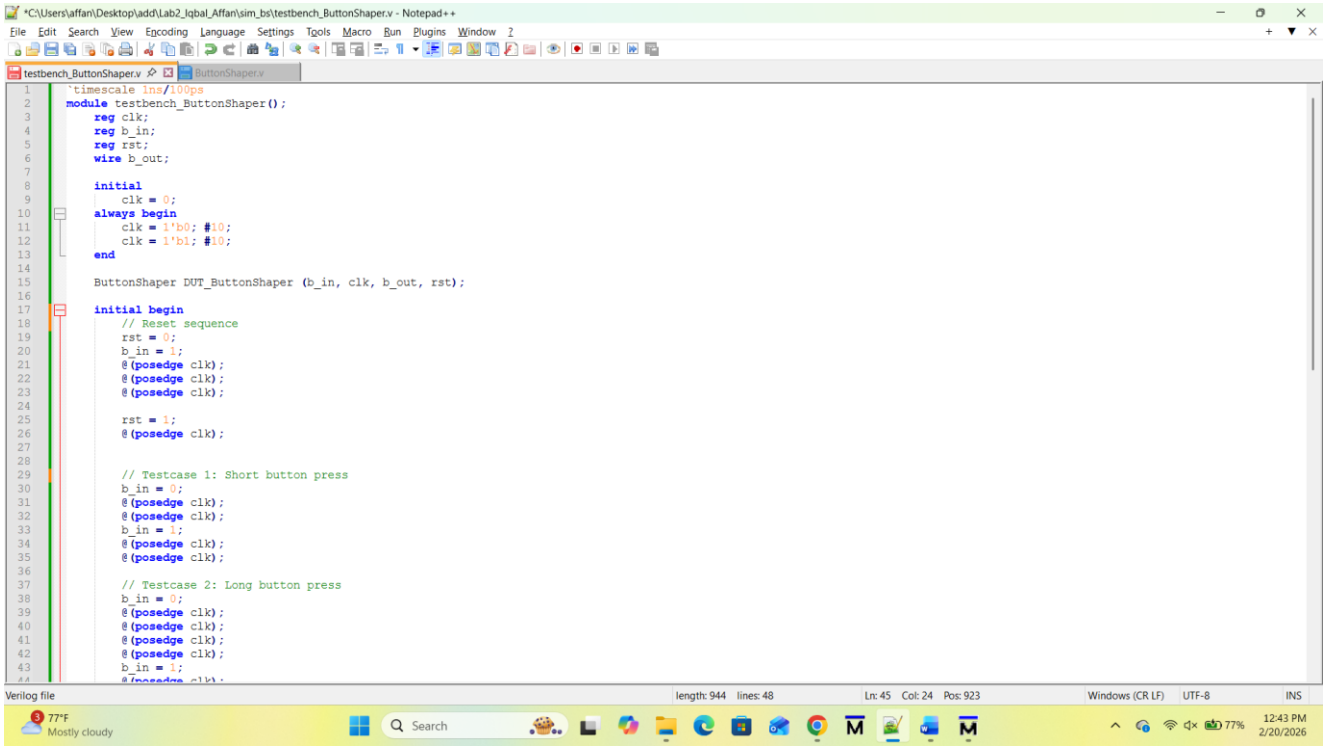


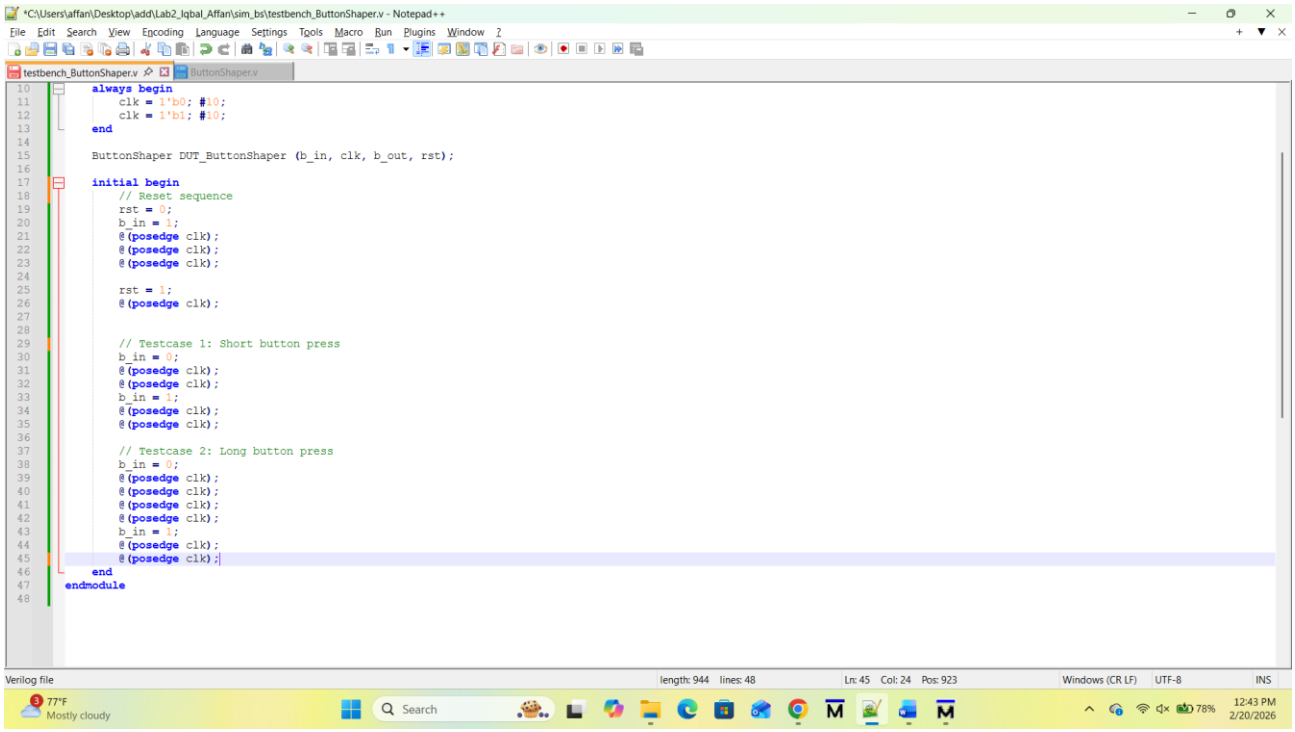
Button Shaper Module

- ***Verilog Module***



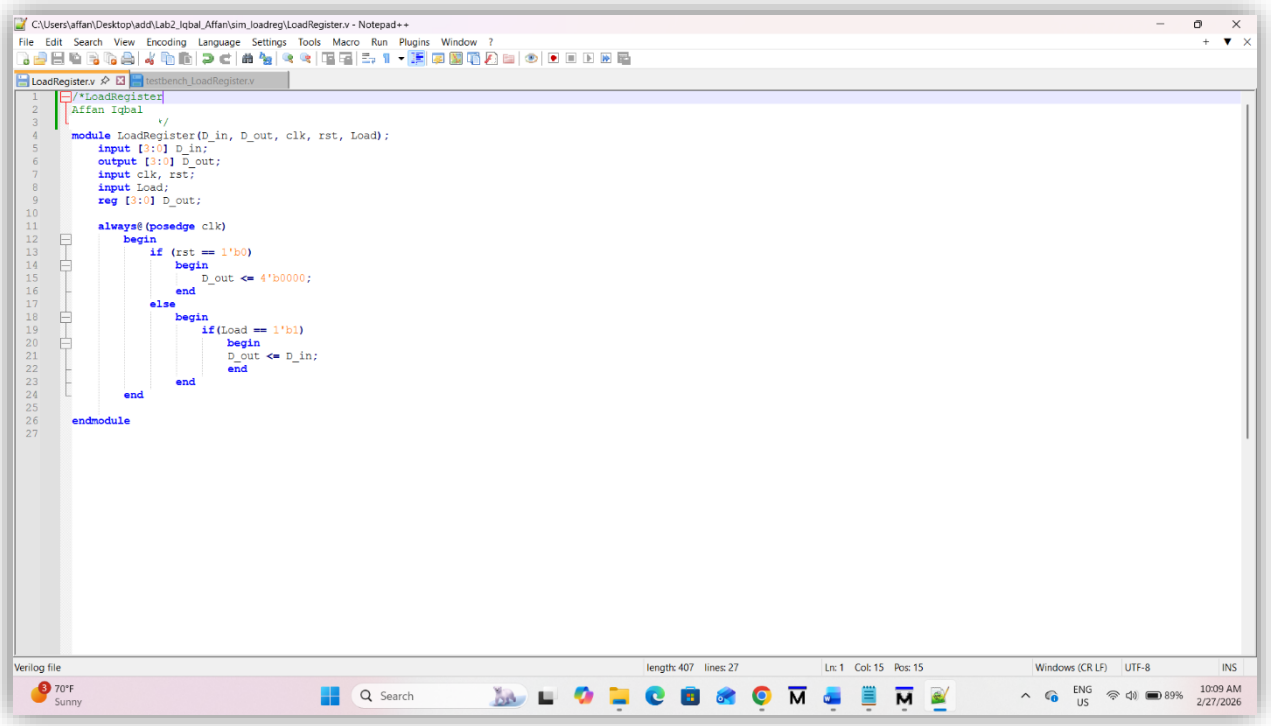
- ***Testbench Module***





Load Register Verilog

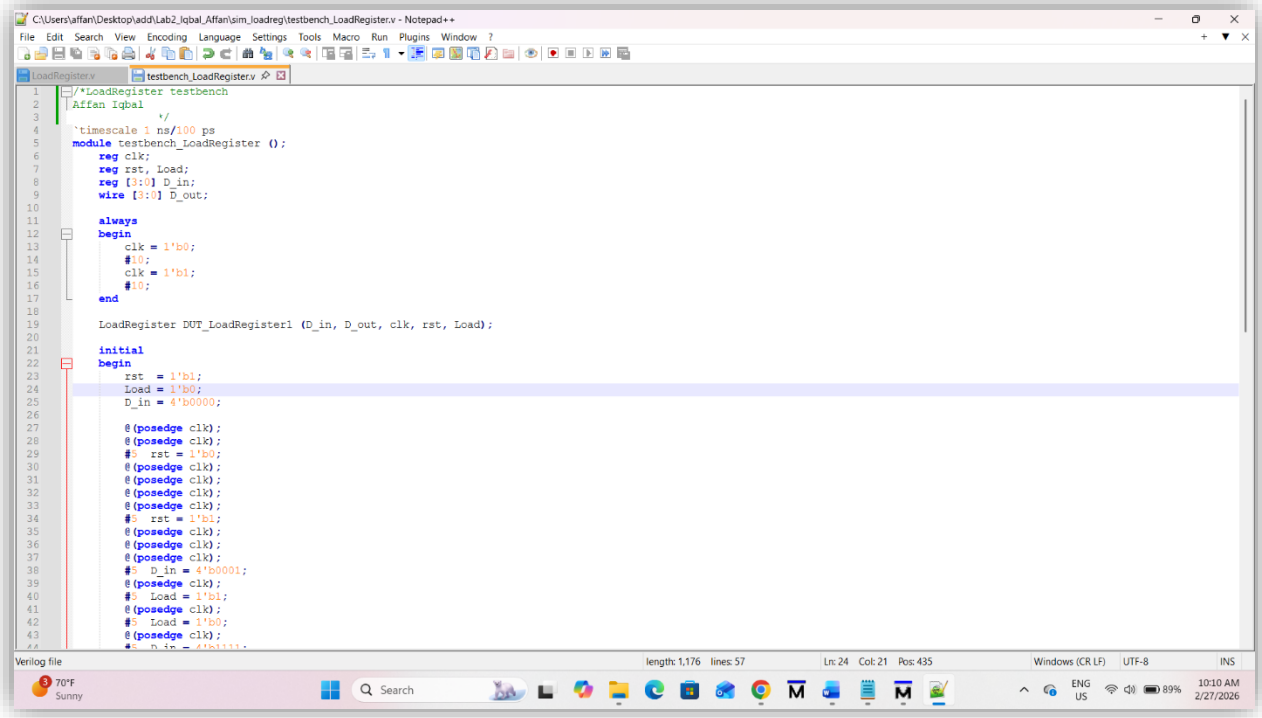
- ***Verilog Module***



```
/*LoadRegister
Affan Iqbal
*/
module LoadRegister(D_in, D_out, clk, rst, Load);
    input [3:0] D_in;
    output [3:0] D_out;
    input clk, rst;
    input Load;
    reg [3:0] D_out;

    always@(posedge clk)
        begin
            if (rst == 1'b0)
                begin
                    D_out <= 4'b0000;
                end
            else
                begin
                    if(Load == 1'b1)
                        begin
                        D_out <= D_in;
                        end
                end
        end

endmodule
```



```
/*LoadRegister testbench
Affan Iqbal
*/
`timescale 1 ns/100 ps
module testbench_LoadRegister ();
    reg clk;
    reg rst, Load;
    reg [3:0] D_in;
    wire [3:0] D_out;

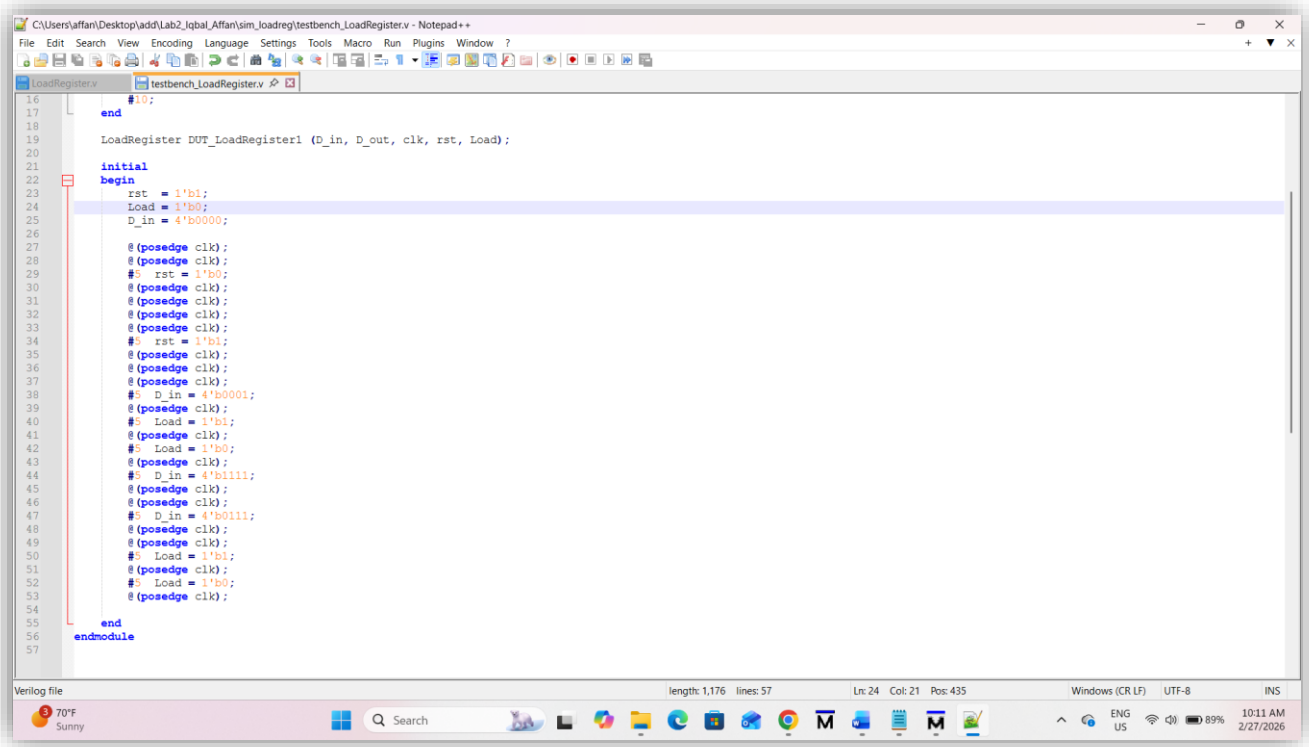
    always
    begin
        clk = 1'b0;
        #10;
        clk = 1'b1;
        #10;
    end

    LoadRegister DUT_LoadRegister1 (D_in, D_out, clk, rst, Load);

    initial
    begin
        rst  = 1'b1;
        Load = 1'b0;
        D_in = 4'b0000;

        @(posedge clk);
        @(posedge clk);
        #5  rst = 1'b0;
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        #5  rst = 1'b1;
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        #5  D_in = 4'b0001;
        @(posedge clk);
        #5  Load = 1'b1;
        @(posedge clk);
        #5  Load = 1'b0;
        @(posedge clk);
```

- ***Testbench Module***



Random Number Generator Verilog

```
/*Number Generator
 Affan Iqbal
              */
module num_gen(
    input  wire clk,
    input  wire rst,
    input  wire rng_gen,
    output reg  [3:0] number
);

    wire feedback;
    assign feedback = number[3] ^ number[2];

    always @(posedge clk) begin
        if (!rst) begin
            number <= 4'd0;
        end
        else begin
            if (!rng_gen) begin
                if (number == 4'd0)
                    number <= 4'b0001;
                else
                    number <= {number[2:0], feedback};
            end
        end
    end

endmodule
```

Testbench for Number Generator

countTo10.v | countTo100.v | digit_timer.v | digit2_timer.v | ms_timer.v | Adder.v | ButtonShaper.v | SevenSegmentDecoder.v | LoadRegister.v | num_gen.v | tb_num_gen.v

```
`timescale 1ns/1ps

module tb_num_gen;

    reg clk = 0;
    reg rst = 0;
    reg rng_gen = 1;
    wire [3:0] number;


    num_gen dut(
        .clk(clk),
        .rst(rst),
        .rng_gen(rng_gen),
        .number(number)
    );

    always #5 clk = ~clk;

    initial begin

        // Apply reset
        rst = 0;
        #20;
        rst = 1;

        rng_gen = 0;
        #200;

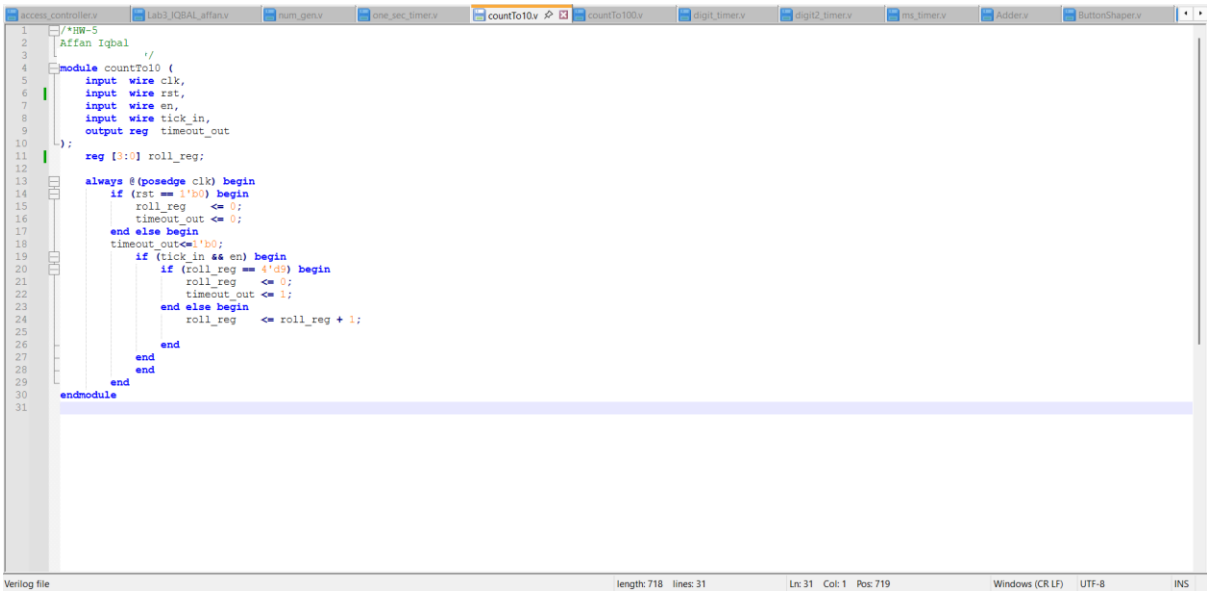
        rng_gen = 1;
        #50;

        rng_gen = 0;
        #200;
    end

endmodule
```

Verilog file | length: 515 lines: 38 | Ln: 17 Col: 1 Pos: 252 | Windows (CR LF) | UTF-8 | INS

CountTo10.v

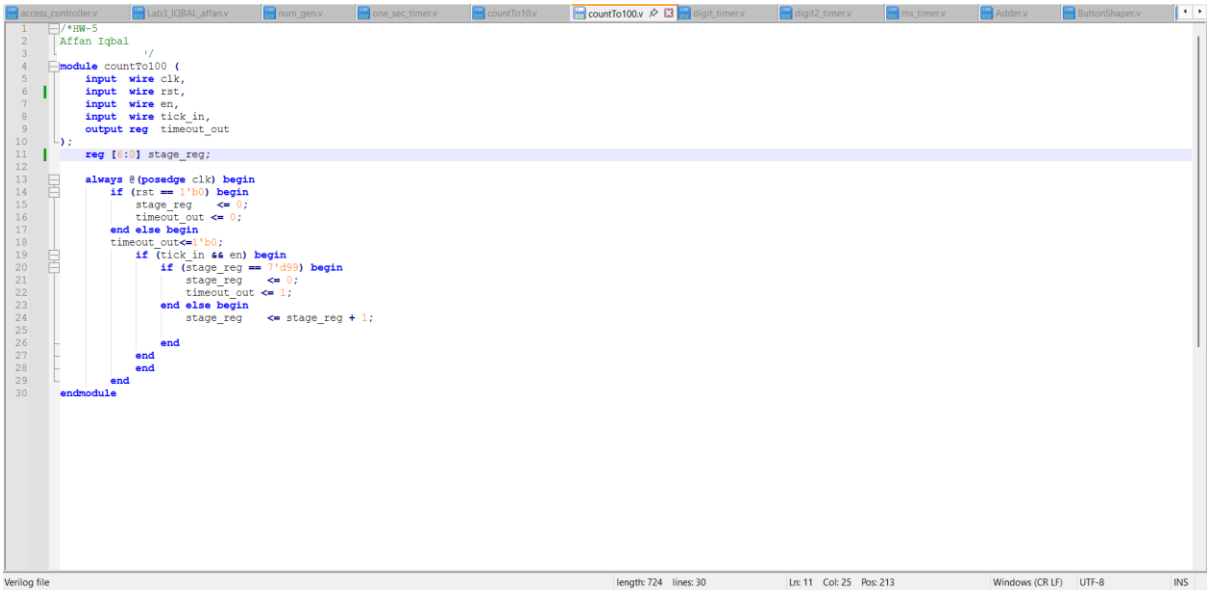


```
/*HW-5
Affan Iqbal
            */
module countTo10 (
    input  wire clk,
    input  wire rst,
    input  wire en,
    input  wire tick_in,
    output reg  timeout_out
);
    reg [3:0] roll_reg;

    always @(posedge clk) begin
        if (rst == 1'b0) begin
            roll_reg    <= 0;
            timeout_out <= 0;
        end else begin
        timeout_out<=1'b0;
            if (tick_in && en) begin
                if (roll_reg == 4'd9) begin
                    roll_reg    <= 0;
                    timeout_out <= 1;
                end else begin
                    roll_reg    <= roll_reg + 1;

                end
            end
            end
        end
endmodule
```

CountTo100.v

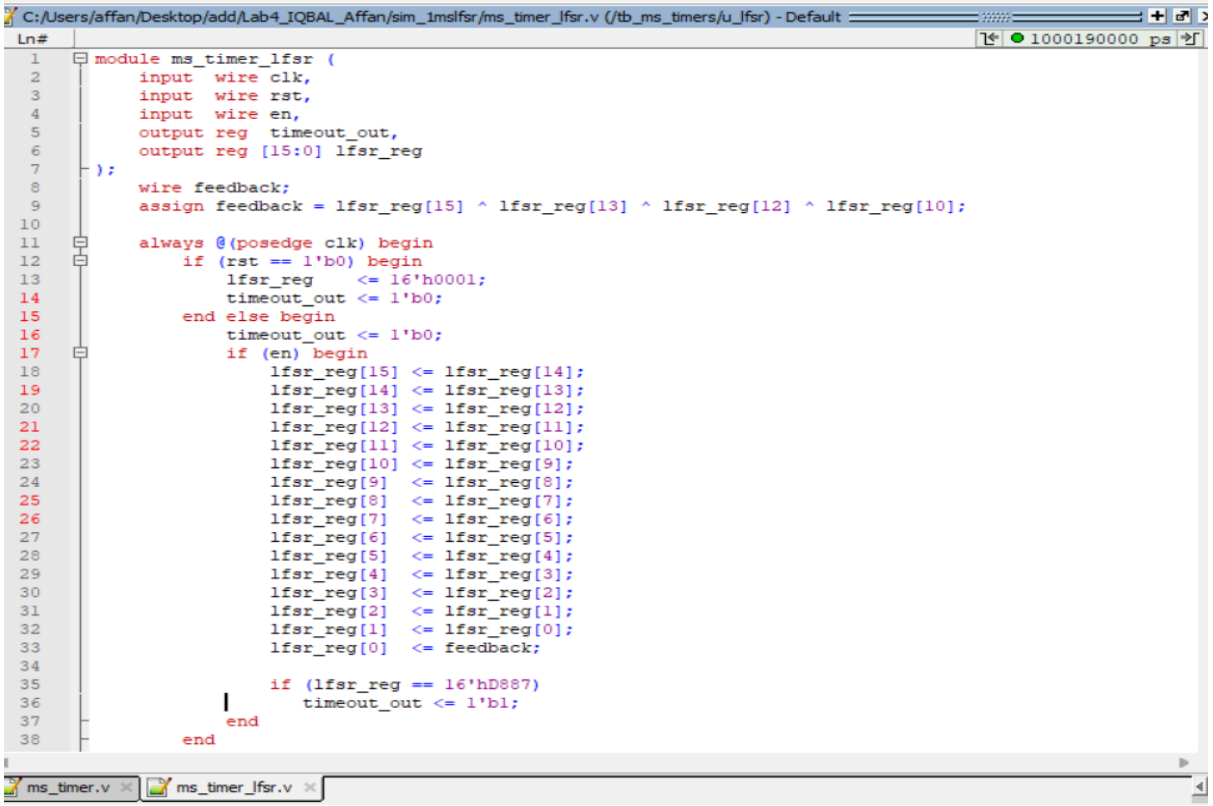


```
/*HW-5
Affan Iqbal
            */
module countTo100 (
    input  wire clk,
    input  wire rst,
    input  wire en,
    input  wire tick_in,
    output reg  timeout_out
);
    reg [6:0] stage_reg;

    always @(posedge clk) begin
        if (rst == 1'b0) begin
            stage_reg    <= 0;
            timeout_out <= 0;
        end else begin
        timeout_out<='b0;
            if (tick_in && en) begin
                if (stage_reg == 7'd99) begin
                    stage_reg    <= 0;
                    timeout_out <= 1;
                end else begin
                    stage_reg    <= stage_reg + 1;

                end
            end
            end
        end
endmodule
```

Ms_timer_lfsr.v based on LFSR

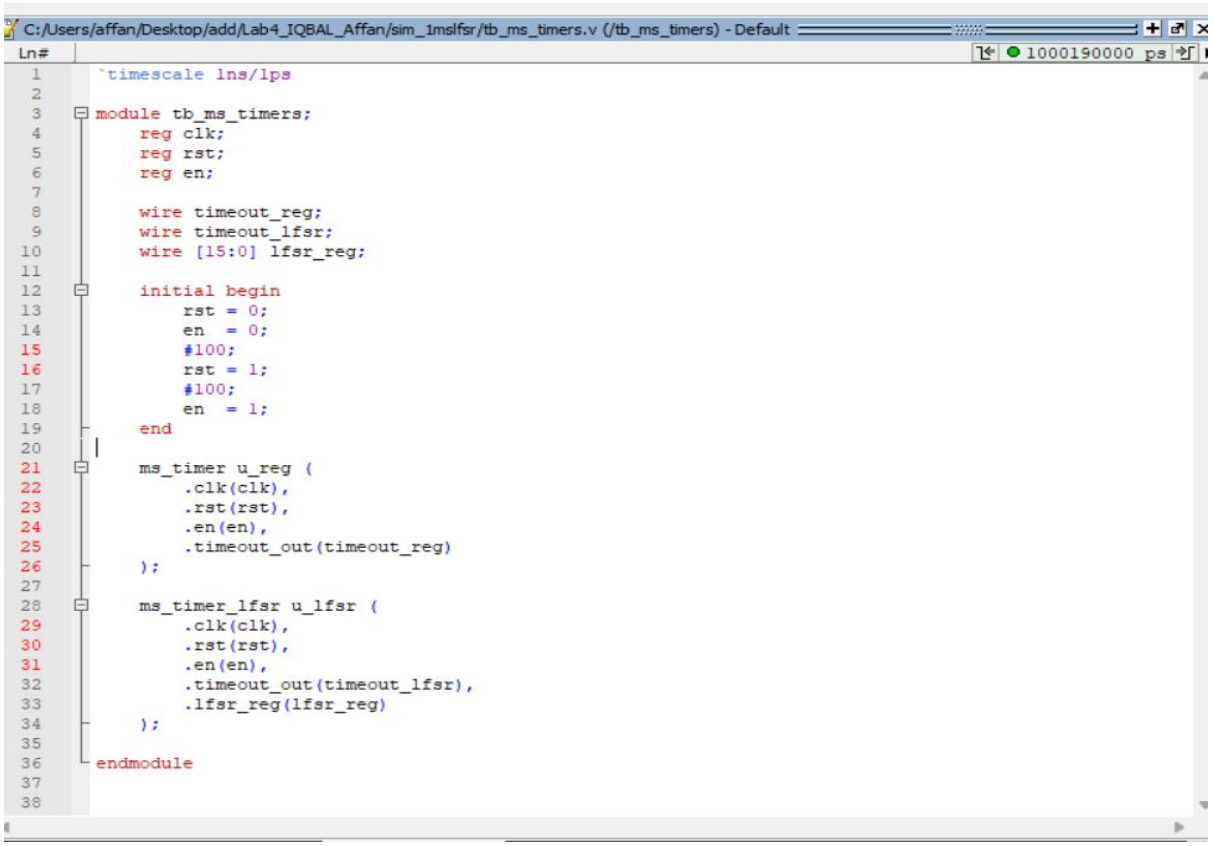



```
module ms_timer_lfsr (
    input  wire clk,
    input  wire rst,
    input  wire en,
    output reg  timeout_out,
    output reg [15:0] lfsr_reg
);
    wire feedback;
    assign feedback = lfsr_reg[15] ^ lfsr_reg[13] ^ lfsr_reg[12] ^ lfsr_reg[10];

    always @(posedge clk) begin
        if (rst == 1'b0) begin
            lfsr_reg    <= 16'h0001;
            timeout_out <= 1'b0;
        end else begin
            timeout_out <= 1'b0;
            if (en) begin
                lfsr_reg[15] <= lfsr_reg[14];
                lfsr_reg[14] <= lfsr_reg[13];
                lfsr_reg[13] <= lfsr_reg[12];
                lfsr_reg[12] <= lfsr_reg[11];
                lfsr_reg[11] <= lfsr_reg[10];
                lfsr_reg[10] <= lfsr_reg[9];
                lfsr_reg[9]  <= lfsr_reg[8];
                lfsr_reg[8]  <= lfsr_reg[7];
                lfsr_reg[7]  <= lfsr_reg[6];
                lfsr_reg[6]  <= lfsr_reg[5];
                lfsr_reg[5]  <= lfsr_reg[4];
                lfsr_reg[4]  <= lfsr_reg[3];
                lfsr_reg[3]  <= lfsr_reg[2];
                lfsr_reg[2]  <= lfsr_reg[1];
                lfsr_reg[1]  <= lfsr_reg[0];
                lfsr_reg[0]  <= feedback;

                if (lfsr_reg == 16'hD887)
                    timeout_out <= 1'b1;
            end
        end
```

Testbench ms_timer_lfsr.v




```
`timescale 1ns/1ps

module tb_ms_timers;
    reg clk;
    reg rst;
    reg en;

    wire timeout_reg;
    wire timeout_lfsr;
    wire [15:0] lfsr_reg;

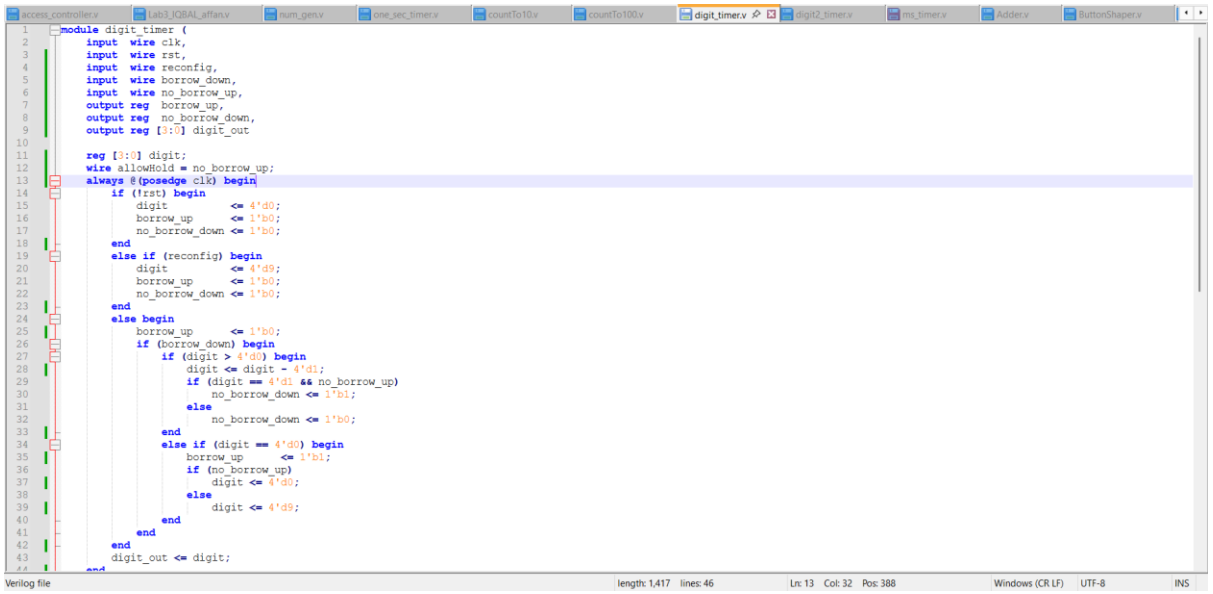
    initial begin
        rst = 0;
        en  = 0;
        #100;
        rst = 1;
        #100;
        en  = 1;
    end

    ms_timer u_reg (
        .clk(clk),
        .rst(rst),
        .en(en),
        .timeout_out(timeout_reg)
    );

    ms_timer_lfsr u_lfsr (
        .clk(clk),
        .rst(rst),
        .en(en),
        .timeout_out(timeout_lfsr),
        .lfsr_reg(lfsr_reg)
    );

endmodule
```

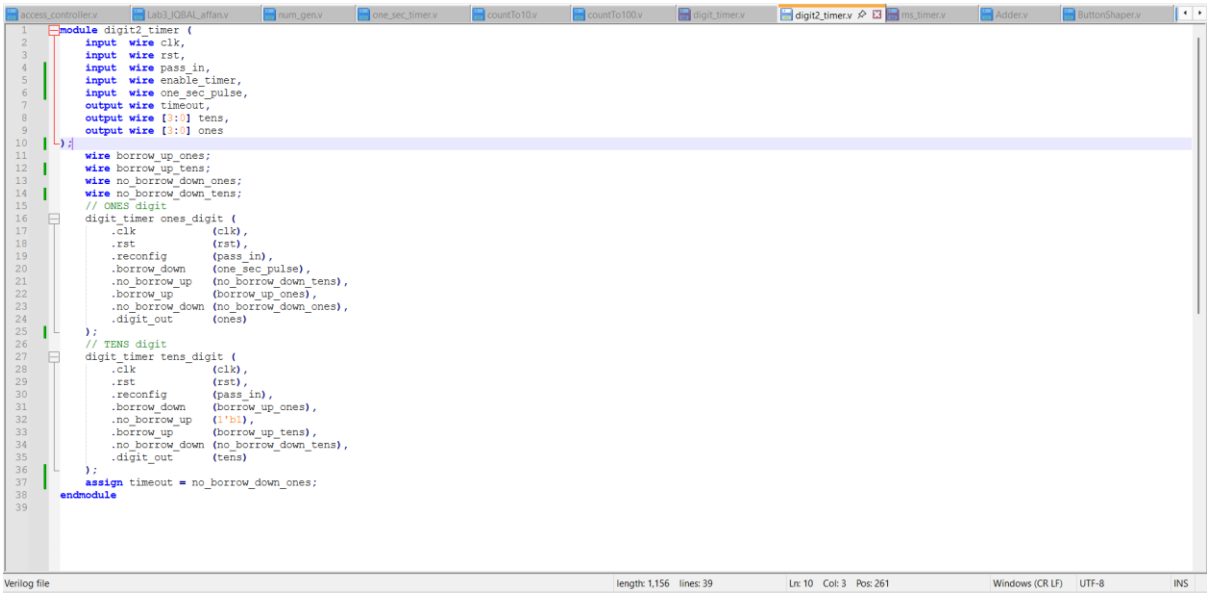
Digit_timer.v




```
module digit_timer (
    input  wire clk,
    input  wire rst,
    input  wire reconfig,
    input  wire borrow_down,
    input  wire no_borrow_up,
    output reg  borrow_up,
    output reg  no_borrow_down,
    output reg [3:0] digit_out

    reg [3:0] digit;
    wire allowHold = no_borrow_up;
    always @(posedge clk) begin
        if (!rst) begin
            digit          <= 4'd0;
            borrow_up      <= 1'b0;
            no_borrow_down <= 1'b0;
        end
        else if (reconfig) begin
            digit          <= 4'd9;
            borrow_up      <= 1'b0;
            no_borrow_down <= 1'b0;
        end
        else begin
            borrow_up      <= 1'b0;
            if (borrow_down) begin
                if (digit > 4'd0) begin
                    digit <= digit - 4'd1;
                    if (digit == 4'd1 && no_borrow_up)
                        no_borrow_down <= 1'b1;
                    else
                        no_borrow_down <= 1'b0;
                end
                else if (digit == 4'd0) begin
                    borrow_up      <= 1'b1;
                    if (no_borrow_up)
                        digit <= 4'd0;
                    else
                        digit <= 4'd9;
                end
            end
        end
        digit_out <= digit;
    end
```

Digit2_timer.v




```
module digit2_timer (
    input  wire clk,
    input  wire rst,
    input  wire pass_in,
    input  wire enable_timer,
    input  wire one_sec_pulse,
    output wire timeout,
    output wire [3:0] tens,
    output wire [3:0] ones
);
    wire borrow_up_ones;
    wire borrow_up_tens;
    wire no_borrow_down_ones;
    wire no_borrow_down_tens;
    // ONES digit
    digit_timer ones_digit (
        .clk            (clk),
        .rst            (rst),
        .reconfig       (pass_in),
        .borrow_down    (one_sec_pulse),
        .no_borrow_up   (no_borrow_down_tens),
        .borrow_up      (borrow_up_ones),
        .no_borrow_down (no_borrow_down_ones),
        .digit_out      (ones)
    );
    // TENS digit
    digit_timer tens_digit (
        .clk            (clk),
        .rst            (rst),
        .reconfig       (pass_in),
        .borrow_down    (borrow_up_ones),
        .no_borrow_up   (1'b1),
        .borrow_up      (borrow_up_tens),
        .no_borrow_down (no_borrow_down_tens),
        .digit_out      (tens)
    );
    assign timeout = no_borrow_down_ones;
endmodule
```

Access Controller Verilog Module

```verilog
module access_controller(
    input       clk,
    input       rst,
    input       enter_pulse,
    input       rng_in,
    input       load_in,
    input  [3:0] digit_in,
    input       timeout,
    input       op,

    output      authenticated,
    output      not_authenticated,
    output      rng_out,
    output      load_out,
    output      timer_reconfig,
    output      timer_enable
);
    wire auth_ok;
    wire auth_fail;

    wire start_game;
    wire game_active;
    wire game_over_sig;
    wire reconfig_sig;


    // ACCESS FSM
    access_fsm ACCESS_CORE (
        .clk(clk),
        .rst(rst),
        .enter_pulse(enter_pulse),
        .digit_in(digit_in),
        .timeout(timeout),

        .authenticated(auth_ok),
        .not_authenticated(auth_fail),

        .start_game(start_game),
        .game_active(game_active),
        .game_over(game_over_sig),
        .reconfig_timer(reconfig_sig)
    );

    // TIMER FSM
    timer_fsm TIMER_CORE (
        .clk(clk),
        .rst(rst),
        .rng_in(rng_in),
```

Verilog file | length: 1,634 | lines: 65 | Ln: 43 | Col: 1 | Pos: 1,101 | Windows (CR LF) | UTF-8 | INS

```verilog
        .enter_pulse(enter_pulse),
        .digit_in(digit_in),
        .timeout(timeout),

        .authenticated(auth_ok),
        .not_authenticated(auth_fail),

        .start_game(start_game),
        .game_active(game_active),
        .game_over(game_over_sig),
        .reconfig_timer(reconfig_sig)
    );

    // TIMER FSM
    timer_fsm TIMER_CORE (
        .clk(clk),
        .rst(rst),
        .rng_in(rng_in),
        .load_in(load_in),

        .start_game(start_game),
        .game_active(game_active),
        .game_over(game_over_sig),
        .reconfig_timer(reconfig_sig),

        .rng_out(rng_out),
        .load_out(load_out),
        .timer_reconfig(timer_reconfig),
        .timer_enable(timer_enable)
    );

    assign authenticated     = auth_ok;
    assign not_authenticated = auth_fail;

endmodule
```

Verilog file | length: 1,634 | lines: 65 | Ln: 43 | Col: 1 | Pos: 1,101 | Windows (CR LF) | UTF-8 | INS

access_fsm_tb.v

```
`timescale 1 ns/100ps

module access_fsm_tb;

    reg clk;
    reg rst;
    reg enter_pulse;
    reg timeout;
    reg [3:0] digit_in;

    wire authenticated;
    wire not_authenticated;
    wire start_game;
    wire game_active;
    wire game_over;
    wire reconfig_timer;

    access_fsm DUT (
        .clk(clk),
        .rst(rst),
        .enter_pulse(enter_pulse),
        .digit_in(digit_in),
        .timeout(timeout),
        .authenticated(authenticated),
        .not_authenticated(not_authenticated),
        .start_game(start_game),
        .game_active(game_active),
        .game_over(game_over),
        .reconfig_timer(reconfig_timer)
    );

    always begin
        clk = 1'b0; #10;
        clk = 1'b1; #10;
    end

    initial begin
        rst = 0;
        enter_pulse = 0;
        timeout = 0;
        digit_in = 0;

        @(posedge clk);
        rst = 1;
        @(posedge clk);

        digit_in = 4'd1;
        enter_pulse = 1; @(posedge clk);
```

```
    initial begin
        rst = 0;
        enter_pulse = 0;
        timeout = 0;
        digit_in = 0;

        @(posedge clk);
        rst = 1;
        @(posedge clk);

        digit_in = 4'd1;
        enter_pulse = 1; @(posedge clk);
        enter_pulse = 0; @(posedge clk);

        digit_in = 4'd3;
        enter_pulse = 1; @(posedge clk);
        enter_pulse = 0; @(posedge clk);

        digit_in = 4'd8;
        enter_pulse = 1; @(posedge clk);
        enter_pulse = 0; @(posedge clk);

        digit_in = 4'd8;
        enter_pulse = 1; @(posedge clk);
        enter_pulse = 0; @(posedge clk);

        repeat(20) @(posedge clk);


        digit_in = 4'd9;
        enter_pulse = 1; @(posedge clk);
        enter_pulse = 0; @(posedge clk);

        digit_in = 4'd9;
        enter_pulse = 1; @(posedge clk);
        enter_pulse = 0; @(posedge clk);

        digit_in = 4'd9;
        enter_pulse = 1; @(posedge clk);
        enter_pulse = 0; @(posedge clk);

        digit_in = 4'd9;
        enter_pulse = 1; @(posedge clk);
        enter_pulse = 0; @(posedge clk);

        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
```

Verilog file | length: 2,082 | lines: 95 | Ln: 91 | Col: 1 | Pos: 2,059 | Windows (CR LF) | UTF-8 | INS

Timer_fsm.v

```
module timer_fsm(
    input  wire clk,
    input  wire rst,

    input  wire go_reconfig,
    input  wire go_wait,
    input  wire go_gameplay,
    input  wire go_gameover,

    output reg  timer_reconfig,
    output reg  timer_enable
);

    always @(posedge clk) begin
        if (!rst) begin
            timer_reconfig <= 0;
            timer_enable   <= 0;
        end
        else begin

            // default
            timer_reconfig <= 0;

            if (go_reconfig) begin
                timer_reconfig <= 1;   // 1-cycle pulse
                timer_enable   <= 0;
            end
            else if (go_wait) begin
                timer_enable <= 0;
            end
            else if (go_gameplay) begin
                timer_enable <= 1;
            end
            else if (go_gameover) begin
                timer_enable <= 0;
            end

        end
    end

endmodule
```

LFSR based One_sec_timer.v

C:/Users/affan/Desktop/add/Lab4_IQBAL_Affan/sim_1mslfsr/ms_timer_lfsr.v (/tb_ms_timers/u_lfsr) - Default

Ln# 1000190000 ps

```
module ms_timer_lfsr (
    input  wire clk,
    input  wire rst,
    input  wire en,
    output reg  timeout_out,
    output reg [15:0] lfsr_reg
);
    wire feedback;
    assign feedback = lfsr_reg[15] ^ lfsr_reg[13] ^ lfsr_reg[12] ^ lfsr_reg[10];

    always @(posedge clk) begin
        if (rst == 1'b0) begin
            lfsr_reg    <= 16'h0001;
            timeout_out <= 1'b0;
        end else begin
            timeout_out <= 1'b0;
            if (en) begin
                lfsr_reg[15] <= lfsr_reg[14];
                lfsr_reg[14] <= lfsr_reg[13];
                lfsr_reg[13] <= lfsr_reg[12];
                lfsr_reg[12] <= lfsr_reg[11];
                lfsr_reg[11] <= lfsr_reg[10];
                lfsr_reg[10] <= lfsr_reg[9];
                lfsr_reg[9]  <= lfsr_reg[8];
                lfsr_reg[8]  <= lfsr_reg[7];
                lfsr_reg[7]  <= lfsr_reg[6];
                lfsr_reg[6]  <= lfsr_reg[5];
                lfsr_reg[5]  <= lfsr_reg[4];
                lfsr_reg[4]  <= lfsr_reg[3];
                lfsr_reg[3]  <= lfsr_reg[2];
                lfsr_reg[2]  <= lfsr_reg[1];
                lfsr_reg[1]  <= lfsr_reg[0];
                lfsr_reg[0]  <= feedback;

                if (lfsr_reg == 16'hD887)
                    timeout_out <= 1'b1;
            end
        end
```

ms_timer.v | ms_timer_lfsr.v

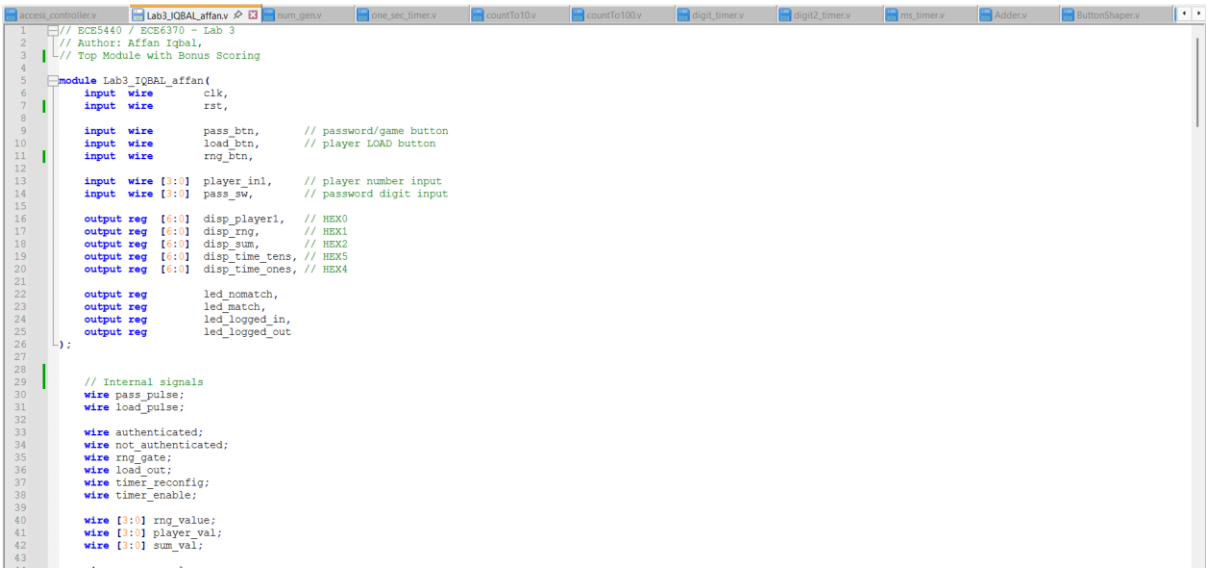
Testbench ms_timer_lfsr

C:/Users/affan/Desktop/add/Lab4_IQBAL_Affan/sim_1mslfsr/tb_ms_timers.v (/tb_ms_timers) - Default

Ln# 1000190000 ps

```
1   `timescale 1ns/1ps
2
3   module tb_ms_timers;
4       reg clk;
5       reg rst;
6       reg en;
7
8       wire timeout_reg;
9       wire timeout_lfsr;
10      wire [15:0] lfsr_reg;
11
12      initial begin
13          rst = 0;
14          en  = 0;
15          #100;
16          rst = 1;
17          #100;
18          en  = 1;
19      end
20
21      ms_timer u_reg (
22          .clk(clk),
23          .rst(rst),
24          .en(en),
25          .timeout_out(timeout_reg)
26      );
27
28      ms_timer_lfsr u_lfsr (
29          .clk(clk),
30          .rst(rst),
31          .en(en),
32          .timeout_out(timeout_lfsr),
33          .lfsr_reg(lfsr_reg)
34      );
35
36  endmodule
37
38
```

Top Module Verilog Code



```
// ECE5440 / ECE6370 – Lab 3
// Author: Affan Iqbal,
// Top Module with Bonus Scoring

module Lab3_IQBAL_affan(
    input  wire        clk,
    input  wire        rst,

    input  wire        pass_btn,      // password/game button
    input  wire        load_btn,      // player LOAD button
    input  wire        rng_btn,

    input  wire [3:0]  player_in1,    // player number input
    input  wire [3:0]  pass_sw,       // password digit input

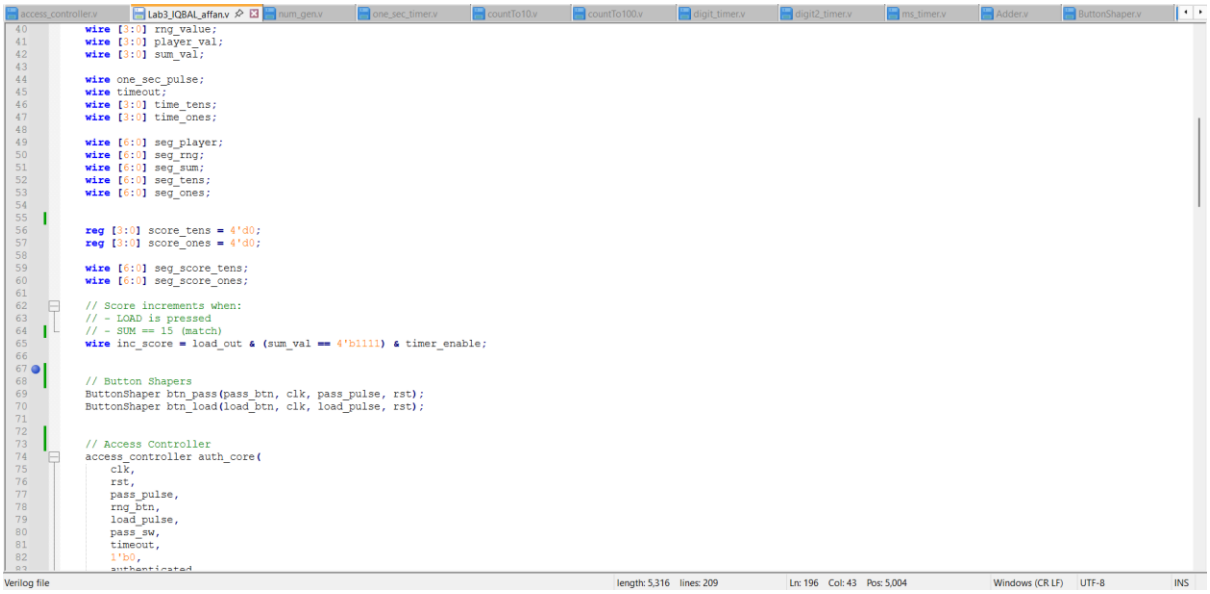
    output reg  [6:0]  disp_player1,  // HEX0
    output reg  [6:0]  disp_rng,      // HEX1
    output reg  [6:0]  disp_sum,      // HEX2
    output reg  [6:0]  disp_time_tens, // HEX5
    output reg  [6:0]  disp_time_ones, // HEX4

    output reg         led_nomatch,
    output reg         led_match,
    output reg         led_logged_in,
    output reg         led_logged_out
);


    // Internal signals
    wire pass_pulse;
    wire load_pulse;

    wire authenticated;
    wire not_authenticated;
    wire rng_gate;
    wire load_out;
    wire timer_reconfig;
    wire timer_enable;

    wire [3:0] rng_value;
    wire [3:0] player_val;
    wire [3:0] sum_val;
```



```
    wire [3:0] rng_value;
    wire [3:0] player_val;
    wire [3:0] sum_val;

    wire one_sec_pulse;
    wire timeout;
    wire [3:0] time_tens;
    wire [3:0] time_ones;

    wire [6:0] seg_player;
    wire [6:0] seg_rng;
    wire [6:0] seg_sum;
    wire [6:0] seg_tens;
    wire [6:0] seg_ones;


    reg [3:0] score_tens = 4'd0;
    reg [3:0] score_ones = 4'd0;

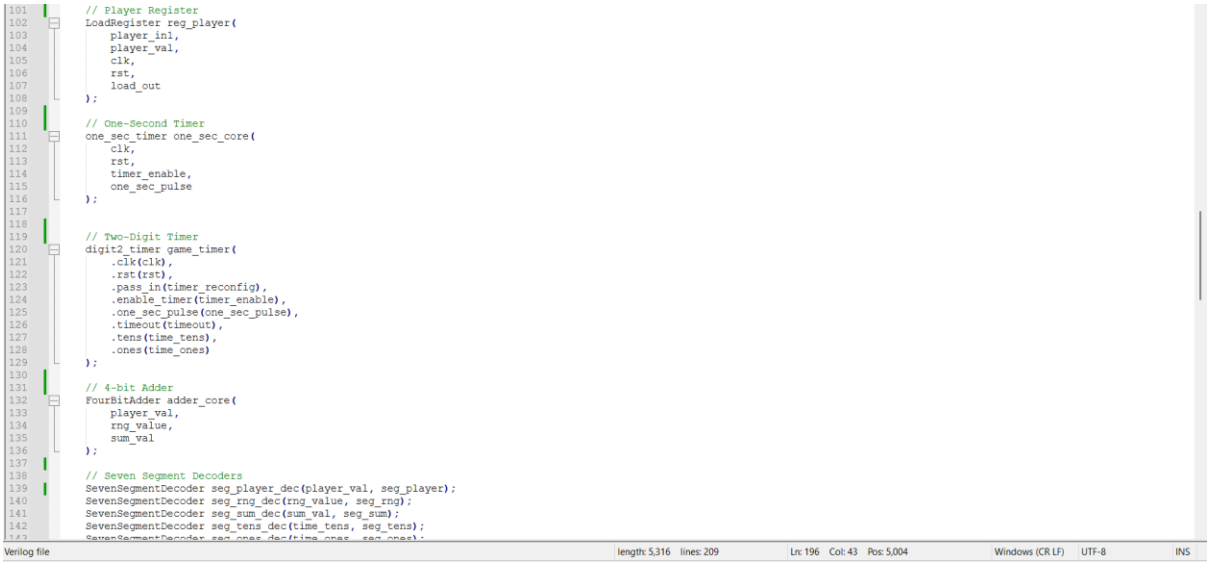
    wire [6:0] seg_score_tens;
    wire [6:0] seg_score_ones;

    // Score increments when:
    // - LOAD is pressed
    // - SUM == 15 (match)
    wire inc_score = load_out & (sum_val == 4'b1111) & timer_enable;


    // Button Shapers
    ButtonShaper btn_pass(pass_btn, clk, pass_pulse, rst);
    ButtonShaper btn_load(load_btn, clk, load_pulse, rst);


    // Access Controller
    access_controller auth_core(
        clk,
        rst,
        pass_pulse,
        rng_btn,
        load_pulse,
        pass_sw,
        timeout,
        1'b0,
```

Verilog file length: 5,316 lines: 209 Ln: 196 Col: 43 Pos: 5,004 Windows (CR LF) UTF-8 INS



```
    // Player Register
    LoadRegister reg_player(
        player_in1,
        player_val,
        clk,
        rst,
        load_out
    );

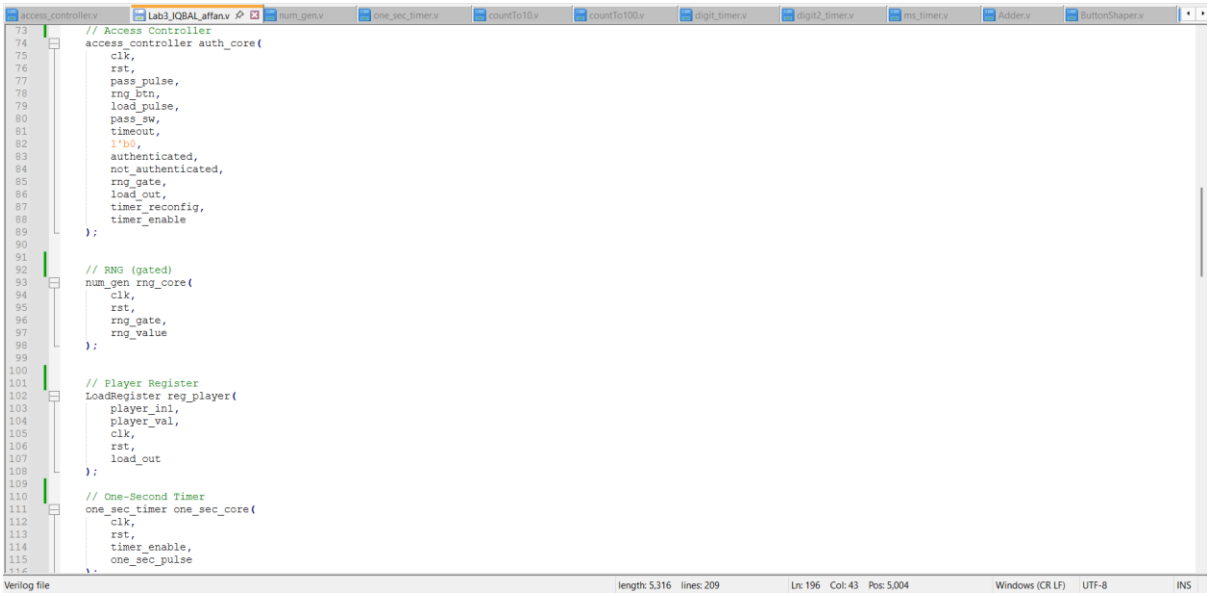
    // One-Second Timer
    one_sec_timer one_sec_core(
        clk,
        rst,
        timer_enable,
        one_sec_pulse
    );


    // Two-Digit Timer
    digit2_timer game_timer(
        .clk(clk),
        .rst(rst),
        .pass_in(timer_reconfig),
        .enable_timer(timer_enable),
        .one_sec_pulse(one_sec_pulse),
        .timeout(timeout),
        .tens(time_tens),
        .ones(time_ones)
    );

    // 4-bit Adder
    FourBitAdder adder_core(
        player_val,
        rng_value,
        sum_val
    );

    // Seven Segment Decoders
    SevenSegmentDecoder seg_player_dec(player_val, seg_player);
    SevenSegmentDecoder seg_rng_dec(rng_value, seg_rng);
    SevenSegmentDecoder seg_sum_dec(sum_val, seg_sum);
    SevenSegmentDecoder seg_tens_dec(time_tens, seg_tens);
```

Verilog file | length: 5,316 lines: 209 | Ln: 196 Col: 43 Pos: 5,004 | Windows (CR LF) | UTF-8 | INS



access_controller.v | Lab3_IQBAL_affan.v | num_gen.v | one_sec_timer.v | countTo10.v | countTo100.v | digit_timer.v | digit2_timer.v | ms_timer.v | Adder.v | ButtonShaper.v

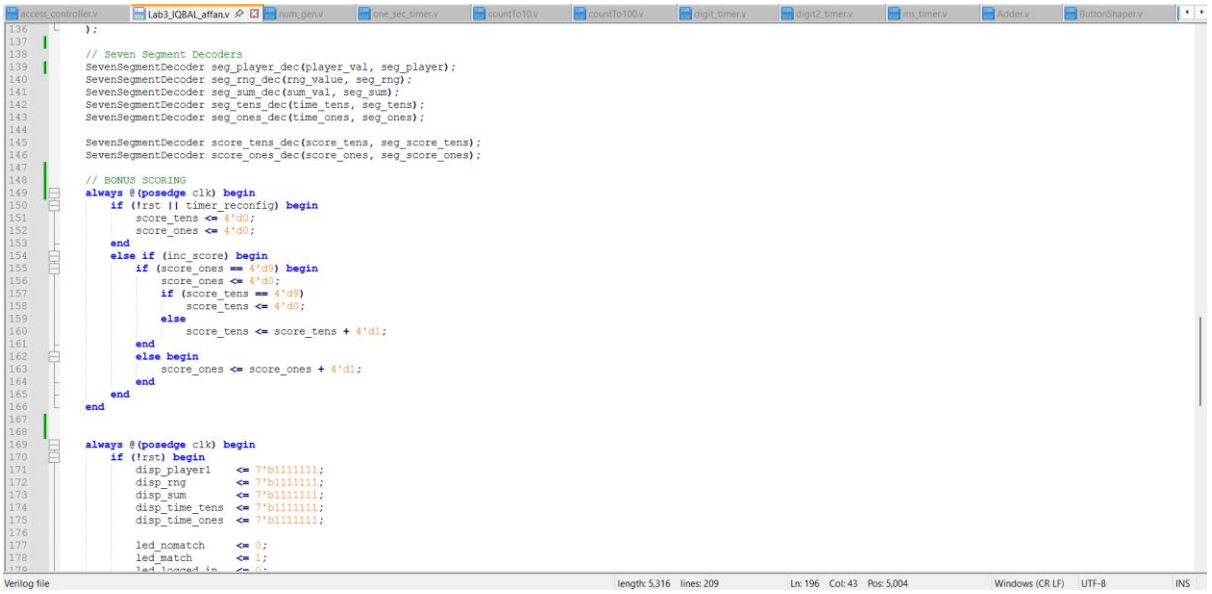
```
    // Access Controller
    access_controller auth_core(
        clk,
        rst,
        pass_pulse,
        rng_btn,
        load_pulse,
        pass_sw,
        timeout,
        1'b0,
        authenticated,
        not_authenticated,
        rng_gate,
        load_out,
        timer_reconfig,
        timer_enable
    );


    // RNG (gated)
    num_gen rng_core(
        clk,
        rst,
        rng_gate,
        rng_value
    );

    // Player Register
    LoadRegister reg_player(
        player_in1,
        player_val,
        clk,
        rst,
        load_out
    );

    // One-Second Timer
    one_sec_timer one_sec_core(
        clk,
        rst,
        timer_enable,
        one_sec_pulse
    );
```

Verilog file | length: 5,316 lines: 209 | Ln: 196 Col: 43 Pos: 5,004 | Windows (CR LF) | UTF-8 | INS



```
    );

    // Seven Segment Decoders
    SevenSegmentDecoder seg_player_dec(player_val, seg_player);
    SevenSegmentDecoder seg_rng_dec(rng_value, seg_rng);
    SevenSegmentDecoder seg_sum_dec(sum_val, seg_sum);
    SevenSegmentDecoder seg_tens_dec(time_tens, seg_tens);
    SevenSegmentDecoder seg_ones_dec(time_ones, seg_ones);

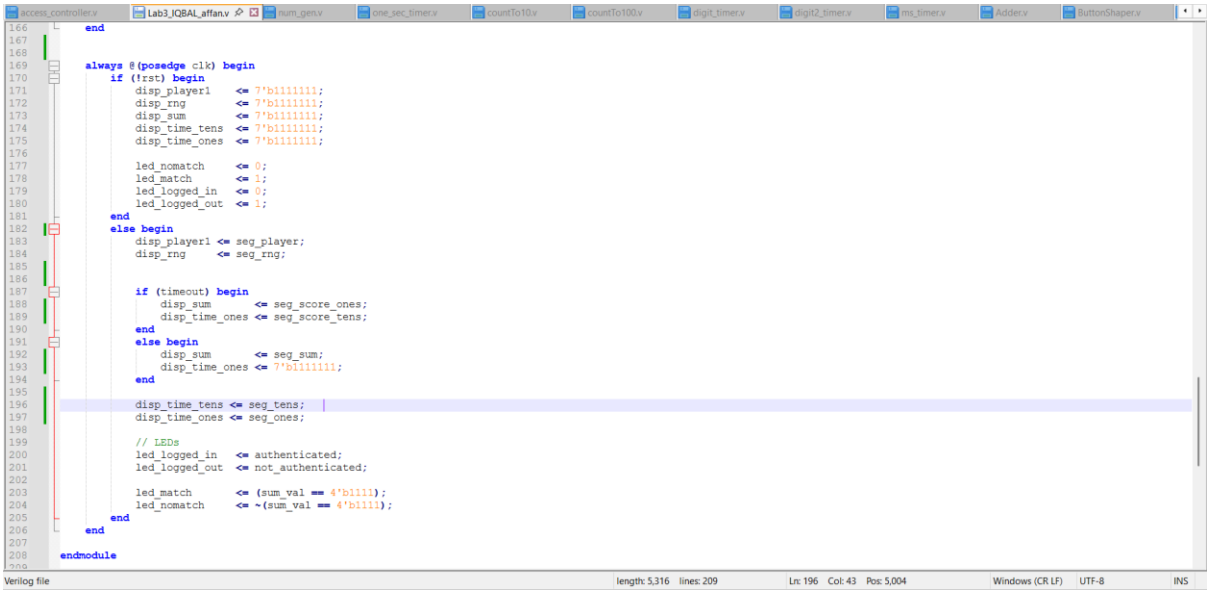
    SevenSegmentDecoder score_tens_dec(score_tens, seg_score_tens);
    SevenSegmentDecoder score_ones_dec(score_ones, seg_score_ones);

    // BONUS SCORING
    always @(posedge clk) begin
        if (!rst || timer_reconfig) begin
            score_tens <= 4'd0;
            score_ones <= 4'd0;
        end
        else if (inc_score) begin
            if (score_ones == 4'd9) begin
                score_ones <= 4'd0;
                if (score_tens == 4'd9)
                    score_tens <= 4'd0;
                else
                    score_tens <= score_tens + 4'd1;
            end
            else begin
                score_ones <= score_ones + 4'd1;
            end
        end
    end


    always @(posedge clk) begin
        if (!rst) begin
            disp_player1    <= 7'b1111111;
            disp_rng        <= 7'b1111111;
            disp_sum        <= 7'b1111111;
            disp_time_tens  <= 7'b1111111;
            disp_time_ones  <= 7'b1111111;

            led_nomatch     <= 0;
            led_match       <= 1;
```

Verilog file | length: 5,316 | lines: 209 | Ln: 196 | Col: 43 | Pos: 5,004 | Windows (CR LF) | UTF-8 | INS



```
    end


    always @(posedge clk) begin
        if (!rst) begin
            disp_player1    <= 7'b1111111;
            disp_rng        <= 7'b1111111;
            disp_sum        <= 7'b1111111;
            disp_time_tens  <= 7'b1111111;
            disp_time_ones  <= 7'b1111111;

            led_nomatch     <= 0;
            led_match       <= 1;
            led_logged_in   <= 0;
            led_logged_out  <= 1;
        end
        else begin
            disp_player1 <= seg_player;
            disp_rng     <= seg_rng;


            if (timeout) begin
                disp_sum       <= seg_score_ones;
                disp_time_ones <= seg_score_tens;
            end
            else begin
                disp_sum       <= seg_sum;
                disp_time_ones <= 7'b1111111;
            end

            disp_time_tens <= seg_tens;
            disp_time_ones <= seg_ones;

            // LEDs
            led_logged_in   <= authenticated;
            led_logged_out  <= not_authenticated;

            led_match       <= (sum_val == 4'b1111);
            led_nomatch     <= ~(sum_val == 4'b1111);
        end
    end


endmodule
```

Verilog file | length: 5,316 | lines: 209 | Ln: 196 | Col: 43 | Pos: 5,004 | Windows (CR LF) | UTF-8 | INS